

Helix Documentation

The Better Gamemode Framework

Generated on January 18, 2026

Source: docs.gethelix.co

Table of Contents

Manual

Clockwork to Helix Migration
Getting Started

Clockwork to Helix Migration
Getting Started

Hooks

Class
Faction

Plugin

Libraries

ix
ix.act
ix.anim
ix.char
ix.chat
ix.class
ix.command
ix.config
ix.currency
ix.data
ix.date

ix.faction
ix.flag
ix.inventory
ix.item
ix.lang
ix.log
ix.menu
ix.notice
ix.option
ix.storage
ix.util

Classes

Character
Entity
Inventory

Item
Player

Panels

ixLabel
ixTooltip

ixTooltipRow

Clockwork to Helix Migration

<https://docs.gethelix.co/manual/convert-from-clockwork/>

Clockwork to Helix Migration

If you are here, you probably want to be converting your code from another framework to Helix. Doing so should not be a difficult task. Most of the previous functions are probably within Helix in one form or another! This means all you need to do is match x function found in the old framework to y function in Helix. Some headings will contain a link - this will bring you to the documentation for Helix's equivalent library or class.

This tutorial assumes basic to intermediate knowledge and experience with Garry's Mod Lua.

Before you start! You will notice that Helix uses `client` for the variable that represents a player. Clockwork uses `player` for the variable instead, but this will conflict with the `player` library. So if you see `_player` being used in Clockwork, it means the Garry's Mod `player` library. This is just a preference and does not affect anything besides `appear`. So keep in mind throughout the tutorial, you may see `player` being used for Clockwork code and `client` being used for Helix code. They represent the same thing, just with a different name.

If you are converting Clockwork code to Helix, keep in mind that `_player` is not defined so you will need to either define `_player` yourself or switch it to `player` instead and change the variable name to `client` for player objects.

Basics of Conversion

Folders

Clockwork code and file structure is not too different from Helix. In the schema, the `plugins` folder and `schema` folder stay in the same place. There are some minor differences in naming however:

- The `schema/entities` folder should be moved outside out of the `schema` folder.
- The `libraries` folder needs to be renamed to `libs` to load.
- The `commands` tab will not load as each command is now defined in a single shared file, does not matter which one.

Deriving from Helix

This is pretty important. If you want to use Helix as the base, you need to set it as the base. So, go to your Clockwork schema's `gamemode` folder. Inside should be two files: `init.lua` and `cl_init.lua`. Open both, and you should see something along the lines of `DeriveGamemode("Clockwork")`. Change this to `DeriveGamemode("helix")`.

The Schema

Introduction

Inside of the `schema` folder of the actual schema, you should see a file named `sh_schema.lua`. This is the main schema file in both Clockwork and Helix. Most of your changes may actually be within this file.

Including Files

Both frameworks come with a utility function to include a file without worrying about sending them to the client and stuff. In Clockwork, this function is

```
Clockwork.kernel:IncludePrefixed("sh_myfile.lua") . Change this to  
ix.util.Include("sh_myfile.lua") and save.
```

The Plugin

Introduction

Plugins serve as a means to add on to a schema or framework without directly modifying either. This allows for easier modifications that can be added/removed with ease. It is recommended that you keep all custom modifications left to plugins rather than editing the framework or the schema if possible.

Structure

All plugins in Clockwork and Helix go into the `plugins` folder. However, there are many differences with the CW plugin structure. First of all, there are two things you see when you open a plugin folder: `plugin` again and `plugin.ini`.

Helix only has one file needed: `sh_plugin.lua` which acts like `sh_schema.lua` but for plugins.

Conversion

The first step is to move all of the contents from the `plugin` folder to the main folder of the plugin folder. The `sh_plugin.lua` file needs to be changed to provide basic information about the plugin. You need to define three things in `sh_plugin.lua` which can be found within the `plugin.ini` file:

- `PLUGIN.name = "Plugin Name"`
- `PLUGIN.author = "Plugin Author"`
- `PLUGIN.description = "Plugin Description"`

If the plugin uses a special variable (e.g. `cwPluginName`) for the plugin, change it to `PLUGIN`.

- Note that the `PLUGIN` table is removed after the plugin is loaded. So if you want to use `PLUGIN` after the plugin has loaded (such as in console commands, in entities, etc.), add `local PLUGIN = PLUGIN` at the top.

- You can see if a global variable is defined for it by looking for `PLUGIN:SetGlobalAlias("cwMyPlugin")`. So, one would change `cwMyPlugin` to `PLUGIN`.

The Character Object

One main thing that is very notable is how the character is referenced using `client:GetCharacter()` which returns a character object. The way the object works is just like an entity you spawn. It has its own properties like the model, color, etc. that makes it unique. You can access all the characters in a table which stores loaded characters with `ix.char.loaded`.

The character object comes with many predefined methods. You can look at how they are defined by [clicking here](#). The character object makes it very simple to manager character information.

You will notice throughout the framework, the character object is used a lot. The use of the character object makes a large barrier between what belongs to the character and what belongs to the player. For example: flags, models, factions, data, and other things are stored on the character and can be accessed by the character object.

In Clockwork, there is no use of an object. Instead, the character information is intertwined with the player object. For example:

```
-- in Clockwork
player:SetCharacterData("foo", "bar")

-- in Helix
client:GetCharacter():SetData("foo", "bar")
```

The use of the character object allows you to access other characters a player might own without needing to have them be the active character, or even access them when the player is not on the server. Overall, the use of the character object may seem like a complex concept, but will simplify a lot of things once you get the hang of the idea.

The Libraries

Animations ([ix.anim](#))

Clockwork features many functions to set up animations for a specific model. Helix too has this functionality. Helix has one function instead that pairs a model to a specific "animation class" (grouping of animation types). So, all one needs to do is find the appropriate animation class to match the model with. Looking at the Clockwork function name should tell you.

```
-- before
Clockwork.animation:AddCivilProtectionModel("models/mymodel.mdl")

-- after
ix.anim.SetModelClass("models/mymodel.mdl", "metrocop")
```

Attributes (`ix.attributes`)

Attributes allow the player to boost certain abilities over time. Both frameworks require one to register attributes, but they are done differently. In Clockwork, the `ATTRIBUTE` table needs to be defined and registered manually. In Helix, the `ATTRIBUTE` table is automatically defined and registered for you. All you need to do is have `ATTRIBUTE.value = "value"`. The basic parts of the attribute needed is `ATTRIBUTE.name` and `ATTRIBUTE.description`.

One extra feature for attributes in Helix is `ATTRIBUTE.OnSetup(client, value)` which is a function that gets called on spawn to apply any effects. For example, the stamina attribute changes the player's run speed by adding the amount of stamina points the player has.

You can find an example at https://github.com/NebulousCloud/helix/blob/master/plugins/stamina/attributes/sh_stm.lua

Classes (`ix.class`)

Classes are a part of the factions. They basically are a more specific form of a faction. Factions in Helix and Clockwork work similarly. For instance, all classes are placed in the `classes` folder under the schema folder and use `CLASS` as the main variable inside the file.

However:

- You do not need to use `local CLASS = Clockwork.class:New("My Class")`. Instead, `CLASS` is already defined for you and you set the name using `CLASS.name = "My Class"`
- `CLASS.factions` is not a table, so `CLASS.factions = {FACTION_MYFACTION}` becomes `CLASS.faction = FACTION_MYFACTION`
- You do not need to use `CLASS:Register()` as classes are registered for you after the file is done processing.
- Classes are optional for factions rather than being required.

Commands (`ix.command`)

Commands no longer need to be in separate files. Instead, they are just placed into one large file. However, if you really wanted you can register multiple commands across multiple files or however you want. One thing you may notice is Clockwork uses a `COMMAND` table while Helix does not always. It is simply a design preference. You can find examples at https://github.com/NebulousCloud/helix/blob/master/gamemode/core/sh_commands.lua

It should be noted that:

- `COMMAND.tip` is not used.
- `COMMAND.text` is not used.
- `COMMAND.flags` is not used.
- `COMMAND.arguments` does not need to be defined if no arguments are needed but is defined as a table of argument types when needed `arguments = {ix.type.character, ix.type.number}`. See [ix.command.CommandArgumentsStructure](#) for details.
- `COMMAND.access` for checking whether or not a person is a (super)admin can be replaced with `adminOnly = true` or `superAdminOnly = true` in the command table.

Configurations ([ix.config](#))

In Helix, the method of adding configurations that can be changed by server owners is heavily simplified. [See an example here](#).

Adding a configuration is as follows:

```
-- before
Clockwork.config:Add("run_speed", 225)

-- after
ix.config.Add("runSpeed", 235, ...)
```

You'll notice that ellipses (...) were added at the end. This is because there are more arguments since adding configuration information has been placed into one function. Additionally:

- `Clockwork.config:ShareKey()` is not needed.
- The 3rd argument for `Clockwork.config:AddToSystem(name, key, description, min, max)` is also the 3rd argument for [ix.config.Add](#)
- The 4th argument for [ix.config.Add](#) is an optional function that is called when the configuration is changed.
- The 5th argument for [ix.config.Add](#) is a table. You can specify the category for the configuration to group it with other configurations. There is also a data table inside which can be used to determine the minimum value and maximum value for numbers. Check out [an example here](#). See also [ix.config](#).

Currency ([ix.currency](#))

Updating your currency code is simple:

```
-- before
Clockwork.config:SetKey("name_cash", "Tokens")
Clockwork.config:SetKey("name_cash", "Dollars") -- another example

-- after
ix.currency.Set("", "token", "tokens")
ix.currency.Set("$", "dollar", "dollars")
```

Note that you need to provide a symbol for that currency (€ for Euro, £ for Pound, ¥ for Yen, etc.) or just leave it as an empty string ("") and then provide the singular form of the name for the currency, then the plural form.

Datastream

Helix uses the [net library](#) whereas Clockwork uses datastream ([netstream](#)).

If you're unfamiliar with the net library, you can include the netstream library to your schema by downloading [netstream](#) to `schema/libs/thirdparty/sh_netstream2.lua` and adding `ix.util.Include("libs/thirdparty/sh_netstream2.lua")` to your `sh_schema.lua` file.

Starting a datastream:


```
-- before
Clockwork.datastream:Start(receiver, "MessageName", {1, 2, 3});

-- after
netstream.Start(receiver, "MessageName", 1, 2, 3)
```

Receiving a datastream:

```
-- before
Clockwork.datastream:Hook("MessageName", function(player, data)
    local a = data[1];
    local b = data[2];
    local c = data[3];

    print(a, b, c);
end);

-- after
netstream.Hook("MessageName", function(client, a, b, c)
    print(a, b, c)
end)
```

Factions ([ix.faction](#))

Factions, like classes, are pretty similar too. They share pretty much the same differences as classes in Clockwork and Helix do.

For instance:

- You do not need to use `local FACTION = Clockwork.faction:New("Name Here")`, instead `FACTION` is already defined for you and you set the name using `FACTION.name = "Name Here"`
- `FACTION.whitelist = true` is changed to `FACTION.isDefault = false`
- `FACTION.models` does not need a male and female part. Instead, all the models are combined into one big list.
- `function FACTION:GetName(name)` becomes `function FACTION:GetDefaultName(name)`
- `FACTION.description = "Describe me"` is added to the faction.
- `FACTION_MYFACTION = FACTION:Register()` becomes `FACTION_MYFACTION = FACTION.index`

Flags ([ix.flag](#))

Flags are functionally equivalent in Helix. To add a new flag:

```
-- before
Clockwork.flag:Add("x", "Name", "Description")

-- after
ix.flag.Add("x", "Description")
```

To check or manipulate a character's flag(s):

```
-- before
Clockwork.player:GiveFlags(player, flags)
Clockwork.player:TakeFlags(player, flags)
Clockwork.player:HasFlags(player, flags)

-- after
client:GetCharacter():GiveFlags(flags)
client:GetCharacter():TakeFlags(flags)
client:GetCharacter():HasFlags(flags)
```

Inventories (Inventory)

Inventories have also had a change in the way they work that may seem very different than Clockwork. Similar to how characters are their own objects, inventories become their own objects as well. These inventory objects belong to character objects, which belongs to players. So, this creates a chain of objects which is neat. The use of inventories as objects makes it very simple to attach inventories to anything.

To access a player's inventory, you need to use `client:GetCharacter():GetInventory()` which returns the main inventory object for the player's character. You can also access all loaded inventories with `ix.item.inventories` but that is not important right now.

Items (Item)

As discussed above, inventories contain items. Items are still used in inventories and world entities, use default class data, have callback functions, and can contain unique item data per instance.

Setting up items

Every time needs to be registered, or have information about it (such as the name, model, what it does, etc.) defined. In Clockwork, you have your items defined in schemas/plugins under the items folder.

So let's start with the differences in structure in the item file.

- `local ITEM = Clockwork.item:New();` is removed
- `ITEM.uniqueID` is completely optional
- Replace `ITEM.cost` with `ITEM.price`
- `ITEM:Register()` is removed

Item Sizes

Helix's inventory uses a grid and utilizes width and height instead of weight as a means of inventory capacity. This means you will have to change your item's weight (`ITEM.weight`) to something that might be analagous to the item's size using `ITEM.width` and `ITEM.height` . The item's size must be at least one by one grid cell. It's up to you to balance the sizes of items in your use case - taking into account how many items a character might have at once, the default inventory size set in the config, etc.

Item Functions

Item functions are defined very differently than they are in Clockwork. For example:

```
-- before
function ITEM:OnUse(player, entity)
    print("My name is: " .. player.Name(), entity)
end

-- after
ITEM.functions.Use = {
    OnRun = function(item)
        print("My name is: " .. item.player, item.entity)
    end
}
```

All item functions are defined in the `ITEM.functions` table. This allows the drop-down menus when using the item a lot easier and cleaner to generate dynamically. There is also more control of the icons used for the options, whether or not the function should be displayed, etc.

You can see an example of a water item here: https://github.com/NebulousCloud/helix-hl2rp/blob/master/schema/items/sh_water.lua

Here, we can define what happens when the function is run, what the icon is, and what sound it plays when used. It is basically put into one area rather than being scattered among hooks and stuff.

Giving/Taking Items

So before we can give/take items, we need to understand what the item instance is. Using the analogy earlier about how the inventory system is like a forum, and inside the forum are posts (the items in this case), we can think of instancing an item as making a new post on a forum. So when we talk about an item instance, it is an item that has been created in the past. The reason we use an item instance (which is its own object too, neat!) is to make each item ever created unique. Each item instance can have its own data unique to itself.

Clockwork also uses an item instance system where you have to instance an item. So, to instance an item in Clockwork you would use:

```
item = Clockwork.item:CreateInstance("item")
```

And this would create a new instance of an item. Helix's instancing system is slightly different. Instead of having the function return the instance like it does in Clockwork, Helix relies on a callback to pass the instance. The reason for this is the item must be inserted into the database to get a unique number to represent that item. This is not done instantly, otherwise servers would freeze when new items are made. Clockwork uses the time and adds a number to get the numeric ID for an item, which allows the item to be returned which "solves" the issue, but I digress.

The Helix equivalent would be:

```
ix.item.Instance(0, "item", data, x, y, function(item) end)
```

Let's break down the differences:

- For Helix's item instance, the 1st argument (`0`) is the inventory that the item belongs to. You can specify `0` so it does not belong to any inventory.
- The data argument is optional and is just a table for the item data.
- `x` and `y` are the position of the items in inventory. You can find an available `x` and `y` with `inventory:FindEmptySlot()`.
- The function is an optional argument that passes the item instance. This is where you can directly access the new item.

Keep in mind that Helix will simplify the item system for you when it can. Normally, you would not need to instance an item yourself unless you were doing something advanced.

So you might be wondering, how do I spawn an item in the map, and how do I give a player an item? In Clockwork, you would do the following:

```
-- spawning an item in the map
Clockwork.entity:CreateItem(player, Clockwork.item:CreateInstance("item"), Vector(1, 2, 3));

-- giving a player an item
player:GiveItem(Clockwork.item:CreateInstance("item"));
```

The equivalent in Helix would be:

```
-- spawning an item in the map
ix.item.Spawn("item", Vector(1, 2, 3))

-- giving a player an item
client:GetCharacter():GetInventory():Add("test")
```

So in these two examples, the whole deal of instancing items is done for you in Helix!

Hooks

You will need to modify the function name and arguments for your schema or plugin hooks.

```
-- before
function Schema:PlayerPlayPainSound(player, gender, damageInfo, hitGroup)
    -- ...
end

-- after
function Schema:GetPlayerPainSound(client)
    -- ...
end
```

You can see the documented hooks for the schema and plugins in the [Plugin](#) section.

Conclusion

Overall, most of the conversion from Clockwork to Helix is simply renaming a certain function and/or switching the order of arguments around. Both are frameworks so they function similarly.

You may want to use our HL2 RP schema example for reference which can be found at <https://github.com/NebulousCloud/helix-hl2rp>

Getting Started

<https://docs.gethelix.co/manual/getting-started/>

Getting Started

It's pretty easy to get started with creating your own schema with Helix. It requires a bit of bootstrapping if you're starting from scratch, but you should quickly be on your way to developing your schema after following one of the below sections in this guide.

Installing the framework

Before you start working on your schema, you'll need to install Helix onto your server. The exact instructions will vary based on your server provider, or if you're hosting the server yourself.

You'll need to download the framework from [GitHub](#) into a folder called `helix`. This folder goes into your server's `gamemodes` folder at `garrysmode/gamemodes/helix`. That's it! The framework is now installed onto your server. Of course, you'll need to restart your server after installing the framework and a schema.

MySQL usage

By default, Helix will use SQLite (which is built into Garry's Mod) to store player/character data. This requires no configuration and will work "out of the box" after installing and will be fine for most server owners. However, you might want to connect your database to your website or use multiple servers with one database - this will require the usage of an external database accessible elsewhere. This will require the use of a MySQL server. Some server providers will provide you with a MySQL database for free to use with your server.

Installing

Helix uses the [MySQLOO](#) library to connect to MySQL databases. You'll need to follow the instructions for installing that library onto your server before continuing. In a nutshell, you need to make sure `gmsv_mysqlOO_win32.dll` or `gmsv_mysqlOO_linux.dll` (depending on your server's operating system) is in the `garrysmode/lua/bin` folder.

In older versions of MySQLOO, you previously required a .dll called `libmysql.dll` to place in your `root` server folder, where `srcds / srcds_linux` was stored. Newer versions of MySQLOO no longer require this.

Configuring

Now that you've installed MySQLOO, you need to tell Helix that you want to connect to an external MySQL database instead of using SQLite. This requires creating a `helix.yml` configuration file in the `garrysmode/gamemodes/helix` folder. There is an example one already made for you called `helix.example.yml` that you can copy and rename to `helix.yml`.

The first thing you'll need to change is the `adapter` entry so that it says `mysqlloo`. Next is to change the other entries to match your database's connection information. Here is an example of what your `helix.yml` should look like:

```
database:
  adapter: "mysqlloo"
  hostname: "myexempldatabase.com"
  username: "myusername"
  password: "mypassword"
  database: "helix"
  port: 3306
```

The `hostname` field can either be a domain name (like `myexempldatabase.com`) or an IP address (`123.123.123.123`). If you don't know what the `port` field should be, simply leave it as the default `3306`; this is the default port for MySQL database connections. The `database` field is the name of the database that you've created for Helix. Note that it does not need to be `helix`, it can be whatever you'd like.

Another important thing to note about this configuration file is that you **must** indent with **two spaces only**. `database` should not have any spacing before it, and all other entries must have two spaces before them. Failing to ensure this will make the configuration file fail to load.

Starting with the HL2 RP schema (Basic)

This section is for using the existing HL2 RP schema as a base for your own schema. It contains a good amount of example code if you need a stronger foundation than just a skeleton.

First, you'll need to download the schema from [GitHub](#). Make sure that you download the contents of the repository into a folder called `ixhl2rp` and place it into your `garrysmo/gamemodes` folder. That's all you'll need to do to get the schema installed, other than setting your gamemode to `ixhl2rp` in the server's command line.

Starting with the skeleton (Basic)

If you don't want excess code you might not use, or prefer to build from an almost-empty foundation that covers the basic bootstrapping, then the skeleton schema is for you. The skeleton schema contains a lot of comments explaining why code is laid out in a certain way, and some other helpful tips/explanations. Make sure you give it a read!

You'll need to download the schema from [GitHub](#) into the folder name of your choice - just make sure it's all lowercase with no spaces. Our example for the sake of brevity will be `myschema`. Place the folder into `garrysmo/gamemodes`.

Next up is to modify the gamemode info so that Garry's Mod will properly recognize it. Rename `skeleton.txt` in your schema folder to your folder's name. In our example we would rename `skeleton.txt` to `myschema.txt`. Next, you'll need to modify the contents of `myschema.txt` and replace the existing information with your own - making sure to replace the `"skeleton"` at the top of the file to your folder's name. In our case we would replace it with `"myschema"`. Once you've renamed the file, you're all good to go!

Converting from Clockwork (Intermediate)

If you are looking to switch to Helix from Clockwork, you can follow the [conversion guide](#).

Starting from scratch (Intermediate)

You can always create the gamemode files yourself if you'd like (although we suggest the skeleton schema in general). In general, a schema is a gamemode that is derived from `helix` and automatically loads `schema/sh_schema.lua`. You shouldn't have your schema files outside of the `schema` folder. The files you'll need are as follows:

`gamemode/init.lua`

```
AddCSLuaFile("cl_init.lua")
DeriveGamemode("helix")
```

`gamemode/cl_init.lua`

```
DeriveGamemode("helix")
```

`schema/sh_schema.lua`

```
Schema.name = "My Schema"
Schema.author = "me!"
Schema.description = "My awesome schema."

-- include your other schema files
ix.util.Include("cl_schema.lua")
ix.util.Include("sv_schema.lua")
-- etc.
```


Clockwork to Helix Migration

<https://docs.gethelix.co/manual/converting-from-clockwork>

Clockwork to Helix Migration

If you are here, you probably want to be converting your code from another framework to Helix. Doing so should not be a difficult task. Most of the previous functions are probably within Helix in one form or another! This means all you need to do is match x function found in the old framework to y function in Helix. Some headings will contain a link - this will bring you to the documentation for Helix's equivalent library or class.

This tutorial assumes basic to intermediate knowledge and experience with Garry's Mod Lua.

Before you start! You will notice that Helix uses `client` for the variable that represents a player. Clockwork uses `player` for the variable instead, but this will conflict with the `player` library. So if you see `_player` being used in Clockwork, it means the Garry's Mod `player` library. This is just a preference and does not affect anything besides `appear`. So keep in mind throughout the tutorial, you may see `player` being used for Clockwork code and `client` being used for Helix code. They represent the same thing, just with a different name.

If you are converting Clockwork code to Helix, keep in mind that `_player` is not defined so you will need to either define `_player` yourself or switch it to `player` instead and change the variable name to `client` for player objects.

Basics of Conversion

Folders

Clockwork code and file structure is not too different from Helix. In the schema, the `plugins` folder and `schema` folder stay in the same place. There are some minor differences in naming however:

- The `schema/entities` folder should be moved outside out of the `schema` folder.
- The `libraries` folder needs to be renamed to `libs` to load.
- The `commands` tab will not load as each command is now defined in a single shared file, does not matter which one.

Deriving from Helix

This is pretty important. If you want to use Helix as the base, you need to set it as the base. So, go to your Clockwork schema's `gamemode` folder. Inside should be two files: `init.lua` and `cl_init.lua`. Open both, and you should see something along the lines of `DeriveGamemode("Clockwork")`. Change this to `DeriveGamemode("helix")`.

The Schema

Introduction

Inside of the `schema` folder of the actual schema, you should see a file named `sh_schema.lua`. This is the main schema file in both Clockwork and Helix. Most of your changes may actually be within this file.

Including Files

Both frameworks come with a utility function to include a file without worrying about sending them to the client and stuff. In Clockwork, this function is

```
Clockwork.kernel:IncludePrefixed("sh_myfile.lua") . Change this to  
ix.util.Include("sh_myfile.lua") and save.
```

The Plugin

Introduction

Plugins serve as a means to add on to a schema or framework without directly modifying either. This allows for easier modifications that can be added/removed with ease. It is recommended that you keep all custom modifications left to plugins rather than editing the framework or the schema if possible.

Structure

All plugins in Clockwork and Helix go into the `plugins` folder. However, there are many differences with the CW plugin structure. First of all, there are two things you see when you open a plugin folder: `plugin` again and `plugin.ini`.

Helix only has one file needed: `sh_plugin.lua` which acts like `sh_schema.lua` but for plugins.

Conversion

The first step is to move all of the contents from the `plugin` folder to the main folder of the plugin folder. The `sh_plugin.lua` file needs to be changed to provide basic information about the plugin. You need to define three things in `sh_plugin.lua` which can be found within the `plugin.ini` file:

- `PLUGIN.name = "Plugin Name"`
- `PLUGIN.author = "Plugin Author"`
- `PLUGIN.description = "Plugin Description"`

If the plugin uses a special variable (e.g. `cwPluginName`) for the plugin, change it to `PLUGIN`.

- Note that the `PLUGIN` table is removed after the plugin is loaded. So if you want to use `PLUGIN` after the plugin has loaded (such as in console commands, in entities, etc.), add `local PLUGIN = PLUGIN` at the top.

- You can see if a global variable is defined for it by looking for `PLUGIN:SetGlobalAlias("cwMyPlugin")`. So, one would change `cwMyPlugin` to `PLUGIN`.

The Character Object

One main thing that is very notable is how the character is referenced using `client:GetCharacter()` which returns a character object. The way the object works is just like an entity you spawn. It has its own properties like the model, color, etc. that makes it unique. You can access all the characters in a table which stores loaded characters with `ix.char.loaded`.

The character object comes with many predefined methods. You can look at how they are defined by [clicking here](#). The character object makes it very simple to manager character information.

You will notice throughout the framework, the character object is used a lot. The use of the character object makes a large barrier between what belongs to the character and what belongs to the player. For example: flags, models, factions, data, and other things are stored on the character and can be accessed by the character object.

In Clockwork, there is no use of an object. Instead, the character information is intertwined with the player object. For example:

```
-- in Clockwork
player:SetCharacterData("foo", "bar")

-- in Helix
client:GetCharacter():SetData("foo", "bar")
```

The use of the character object allows you to access other characters a player might own without needing to have them be the active character, or even access them when the player is not on the server. Overall, the use of the character object may seem like a complex concept, but will simplify a lot of things once you get the hang of the idea.

The Libraries

Animations ([ix.anim](#))

Clockwork features many functions to set up animations for a specific model. Helix too has this functionality. Helix has one function instead that pairs a model to a specific "animation class" (grouping of animation types). So, all one needs to do is find the appropriate animation class to match the model with. Looking at the Clockwork function name should tell you.

```
-- before
Clockwork.animation:AddCivilProtectionModel("models/mymodel.mdl")

-- after
ix.anim.SetModelClass("models/mymodel.mdl", "metrocop")
```

Attributes (`ix.attributes`)

Attributes allow the player to boost certain abilities over time. Both frameworks require one to register attributes, but they are done differently. In Clockwork, the `ATTRIBUTE` table needs to be defined and registered manually. In Helix, the `ATTRIBUTE` table is automatically defined and registered for you. All you need to do is have `ATTRIBUTE.value = "value"`. The basic parts of the attribute needed is `ATTRIBUTE.name` and `ATTRIBUTE.description`.

One extra feature for attributes in Helix is `ATTRIBUTE.OnSetup(client, value)` which is a function that gets called on spawn to apply any effects. For example, the stamina attribute changes the player's run speed by adding the amount of stamina points the player has.

You can find an example at https://github.com/NebulousCloud/helix/blob/master/plugins/stamina/attributes/sh_stm.lua

Classes (`ix.class`)

Classes are a part of the factions. They basically are a more specific form of a faction. Factions in Helix and Clockwork work similarly. For instance, all classes are placed in the `classes` folder under the schema folder and use `CLASS` as the main variable inside the file.

However:

- You do not need to use `local CLASS = Clockwork.class:New("My Class")`. Instead, `CLASS` is already defined for you and you set the name using `CLASS.name = "My Class"`
- `CLASS.factions` is not a table, so `CLASS.factions = {FACTION_MYFACTION}` becomes `CLASS.faction = FACTION_MYFACTION`
- You do not need to use `CLASS:Register()` as classes are registered for you after the file is done processing.
- Classes are optional for factions rather than being required.

Commands (`ix.command`)

Commands no longer need to be in separate files. Instead, they are just placed into one large file. However, if you really wanted you can register multiple commands across multiple files or however you want. One thing you may notice is Clockwork uses a `COMMAND` table while Helix does not always. It is simply a design preference. You can find examples at https://github.com/NebulousCloud/helix/blob/master/gamemode/core/sh_commands.lua

It should be noted that:

- `COMMAND.tip` is not used.
- `COMMAND.text` is not used.
- `COMMAND.flags` is not used.
- `COMMAND.arguments` does not need to be defined if no arguments are needed but is defined as a table of argument types when needed `arguments = {ix.type.character, ix.type.number}`. See [ix.command.CommandArgumentsStructure](#) for details.
- `COMMAND.access` for checking whether or not a person is a (super)admin can be replaced with `adminOnly = true` or `superAdminOnly = true` in the command table.

Configurations ([ix.config](#))

In Helix, the method of adding configurations that can be changed by server owners is heavily simplified. [See an example here](#).

Adding a configuration is as follows:

```
-- before
Clockwork.config:Add("run_speed", 225)

-- after
ix.config.Add("runSpeed", 235, ...)
```

You'll notice that ellipses (...) were added at the end. This is because there are more arguments since adding configuration information has been placed into one function. Additionally:

- `Clockwork.config:ShareKey()` is not needed.
- The 3rd argument for `Clockwork.config:AddToSystem(name, key, description, min, max)` is also the 3rd argument for [ix.config.Add](#)
- The 4th argument for [ix.config.Add](#) is an optional function that is called when the configuration is changed.
- The 5th argument for [ix.config.Add](#) is a table. You can specify the category for the configuration to group it with other configurations. There is also a data table inside which can be used to determine the minimum value and maximum value for numbers. Check out [an example here](#). See also [ix.config](#).

Currency ([ix.currency](#))

Updating your currency code is simple:

```
-- before
Clockwork.config:SetKey("name_cash", "Tokens")
Clockwork.config:SetKey("name_cash", "Dollars") -- another example

-- after
ix.currency.Set("", "token", "tokens")
ix.currency.Set("$", "dollar", "dollars")
```

Note that you need to provide a symbol for that currency (€ for Euro, £ for Pound, ¥ for Yen, etc.) or just leave it as an empty string ("") and then provide the singular form of the name for the currency, then the plural form.

Datastream

Helix uses the [net library](#) whereas Clockwork uses datastream ([netstream](#)).

If you're unfamiliar with the net library, you can include the netstream library to your schema by downloading [netstream](#) to `schema/libs/thirdparty/sh_netstream2.lua` and adding `ix.util.Include("libs/thirdparty/sh_netstream2.lua")` to your `sh_schema.lua` file.

Starting a datastream:

```
-- before
Clockwork.datastream:Start(receiver, "MessageName", {1, 2, 3});

-- after
netstream.Start(receiver, "MessageName", 1, 2, 3)
```

Receiving a datastream:

```
-- before
Clockwork.datastream:Hook("MessageName", function(player, data)
    local a = data[1];
    local b = data[2];
    local c = data[3];

    print(a, b, c);
end);

-- after
netstream.Hook("MessageName", function(client, a, b, c)
    print(a, b, c)
end)
```

Factions ([ix.faction](#))

Factions, like classes, are pretty similar too. They share pretty much the same differences as classes in Clockwork and Helix do.

For instance:

- You do not need to use `local FACTION = Clockwork.faction:New("Name Here")`, instead `FACTION` is already defined for you and you set the name using `FACTION.name = "Name Here"`
- `FACTION.whitelist = true` is changed to `FACTION.isDefault = false`
- `FACTION.models` does not need a male and female part. Instead, all the models are combined into one big list.
- `function FACTION:GetName(name)` becomes `function FACTION:GetDefaultName(name)`
- `FACTION.description = "Describe me"` is added to the faction.
- `FACTION_MYFACTION = FACTION:Register()` becomes `FACTION_MYFACTION = FACTION.index`

Flags ([ix.flag](#))

Flags are functionally equivalent in Helix. To add a new flag:

```
-- before
Clockwork.flag:Add("x", "Name", "Description")

-- after
ix.flag.Add("x", "Description")
```

To check or manipulate a character's flag(s):

```
-- before
Clockwork.player:GiveFlags(player, flags)
Clockwork.player:TakeFlags(player, flags)
Clockwork.player:HasFlags(player, flags)

-- after
client:GetCharacter():GiveFlags(flags)
client:GetCharacter():TakeFlags(flags)
client:GetCharacter():HasFlags(flags)
```

Inventories (Inventory)

Inventories have also had a change in the way they work that may seem very different than Clockwork. Similar to how characters are their own objects, inventories become their own objects as well. These inventory objects belong to character objects, which belongs to players. So, this creates a chain of objects which is neat. The use of inventories as objects makes it very simple to attach inventories to anything.

To access a player's inventory, you need to use `client:GetCharacter():GetInventory()` which returns the main inventory object for the player's character. You can also access all loaded inventories with `ix.item.inventories` but that is not important right now.

Items (Item)

As discussed above, inventories contain items. Items are still used in inventories and world entities, use default class data, have callback functions, and can contain unique item data per instance.

Setting up items

Every time needs to be registered, or have information about it (such as the name, model, what it does, etc.) defined. In Clockwork, you have your items defined in schemas/plugins under the items folder.

So let's start with the differences in structure in the item file.

- `local ITEM = Clockwork.item:New();` is removed
- `ITEM.uniqueID` is completely optional
- Replace `ITEM.cost` with `ITEM.price`
- `ITEM:Register()` is removed

Item Sizes

Helix's inventory uses a grid and utilizes width and height instead of weight as a means of inventory capacity. This means you will have to change your item's weight (`ITEM.weight`) to something that might be analagous to the item's size using `ITEM.width` and `ITEM.height` . The item's size must be at least one by one grid cell. It's up to you to balance the sizes of items in your use case - taking into account how many items a character might have at once, the default inventory size set in the config, etc.

Item Functions

Item functions are defined very differently than they are in Clockwork. For example:

```
-- before
function ITEM:OnUse(player, entity)
    print("My name is: " .. player.Name(), entity)
end

-- after
ITEM.functions.Use = {
    OnRun = function(item)
        print("My name is: " .. item.player, item.entity)
    end
}
```

All item functions are defined in the `ITEM.functions` table. This allows the drop-down menus when using the item a lot easier and cleaner to generate dynamically. There is also more control of the icons used for the options, whether or not the function should be displayed, etc.

You can see an example of a water item here: https://github.com/NebulousCloud/helix-hl2rp/blob/master/schema/items/sh_water.lua

Here, we can define what happens when the function is run, what the icon is, and what sound it plays when used. It is basically put into one area rather than being scattered among hooks and stuff.

Giving/Taking Items

So before we can give/take items, we need to understand what the item instance is. Using the analogy earlier about how the inventory system is like a forum, and inside the forum are posts (the items in this case), we can think of instancing an item as making a new post on a forum. So when we talk about an item instance, it is an item that has been created in the past. The reason we use an item instance (which is its own object too, neat!) is to make each item ever created unique. Each item instance can have its own data unique to itself.

Clockwork also uses an item instance system where you have to instance an item. So, to instance an item in Clockwork you would use:

```
item = Clockwork.item:CreateInstance("item")
```

And this would create a new instance of an item. Helix's instancing system is slightly different. Instead of having the function return the instance like it does in Clockwork, Helix relies on a callback to pass the instance. The reason for this is the item must be inserted into the database to get a unique number to represent that item. This is not done instantly, otherwise servers would freeze when new items are made. Clockwork uses the time and adds a number to get the numeric ID for an item, which allows the item to be returned which "solves" the issue, but I digress.

The Helix equivalent would be:

```
ix.item.Instance(0, "item", data, x, y, function(item) end)
```


Let's break down the differences:

- For Helix's item instance, the 1st argument (`0`) is the inventory that the item belongs to. You can specify `0` so it does not belong to any inventory.
- The data argument is optional and is just a table for the item data.
- `x` and `y` are the position of the items in inventory. You can find an available `x` and `y` with `inventory:FindEmptySlot()`.
- The function is an optional argument that passes the item instance. This is where you can directly access the new item.

Keep in mind that Helix will simplify the item system for you when it can. Normally, you would not need to instance an item yourself unless you were doing something advanced.

So you might be wondering, how do I spawn an item in the map, and how do I give a player an item? In Clockwork, you would do the following:

```
-- spawning an item in the map
Clockwork.entity:CreateItem(player, Clockwork.item:CreateInstance("item"), Vector(1, 2, 3));

-- giving a player an item
player:GiveItem(Clockwork.item:CreateInstance("item"));
```

The equivalent in Helix would be:

```
-- spawning an item in the map
ix.item.Spawn("item", Vector(1, 2, 3))

-- giving a player an item
client:GetCharacter():GetInventory():Add("test")
```

So in these two examples, the whole deal of instancing items is done for you in Helix!

Hooks

You will need to modify the function name and arguments for your schema or plugin hooks.

```
-- before
function Schema:PlayerPlayPainSound(player, gender, damageInfo, hitGroup)
    -- ...
end

-- after
function Schema:GetPlayerPainSound(client)
    -- ...
end
```

You can see the documented hooks for the schema and plugins in the [Plugin](#) section.

Conclusion

Overall, most of the conversion from Clockwork to Helix is simply renaming a certain function and/or switching the order of arguments around. Both are frameworks so they function similarly.

You may want to use our HL2 RP schema example for reference which can be found at <https://github.com/NebulousCloud/helix-hl2rp>

Getting Started

<https://docs.gethelix.co/manual/getting-started>

Getting Started

It's pretty easy to get started with creating your own schema with Helix. It requires a bit of bootstrapping if you're starting from scratch, but you should quickly be on your way to developing your schema after following one of the below sections in this guide.

Installing the framework

Before you start working on your schema, you'll need to install Helix onto your server. The exact instructions will vary based on your server provider, or if you're hosting the server yourself.

You'll need to download the framework from [GitHub](#) into a folder called `helix`. This folder goes into your server's `gamemodes` folder at `garrysmode/gamemodes/helix`. That's it! The framework is now installed onto your server. Of course, you'll need to restart your server after installing the framework and a schema.

MySQL usage

By default, Helix will use SQLite (which is built into Garry's Mod) to store player/character data. This requires no configuration and will work "out of the box" after installing and will be fine for most server owners. However, you might want to connect your database to your website or use multiple servers with one database - this will require the usage of an external database accessible elsewhere. This will require the use of a MySQL server. Some server providers will provide you with a MySQL database for free to use with your server.

Installing

Helix uses the [MySQLOO](#) library to connect to MySQL databases. You'll need to follow the instructions for installing that library onto your server before continuing. In a nutshell, you need to make sure `gmsv_mysqlloo_win32.dll` or `gmsv_mysqlloo_linux.dll` (depending on your server's operating system) is in the `garrysmode/lua/bin` folder.

In older versions of MySQLOO, you previously required a `.dll` called `libmysql.dll` to place in your `root` server folder, where `srcds / srcds_linux` was stored. Newer versions of MySQLOO no longer require this.

Configuring

Now that you've installed MySQLOO, you need to tell Helix that you want to connect to an external MySQL database instead of using SQLite. This requires creating a `helix.yml` configuration file in the `garrysmode/gamemodes/helix` folder. There is an example one already made for you called `helix.example.yml` that you can copy and rename to `helix.yml`.

The first thing you'll need to change is the `adapter` entry so that it says `mysqlloo`. Next is to change the other entries to match your database's connection information. Here is an example of what your `helix.yml` should look like:

```
database:
  adapter: "mysqlloo"
  hostname: "myexampledatabase.com"
  username: "myusername"
  password: "mypassword"
  database: "helix"
  port: 3306
```

The `hostname` field can either be a domain name (like `myexampledatabase.com`) or an IP address (`123.123.123.123`). If you don't know what the `port` field should be, simply leave it as the default `3306`; this is the default port for MySQL database connections. The `database` field is the name of the database that you've created for Helix. Note that it does not need to be `helix`, it can be whatever you'd like.

Another important thing to note about this configuration file is that you **must** indent with **two spaces only**. `database` should not have any spacing before it, and all other entries must have two spaces before them. Failing to ensure this will make the configuration file fail to load.

Starting with the HL2 RP schema (Basic)

This section is for using the existing HL2 RP schema as a base for your own schema. It contains a good amount of example code if you need a stronger foundation than just a skeleton.

First, you'll need to download the schema from [GitHub](#). Make sure that you download the contents of the repository into a folder called `ixhl2rp` and place it into your `garrysmo/gamemodes` folder. That's all you'll need to do to get the schema installed, other than setting your gamemode to `ixhl2rp` in the server's command line.

Starting with the skeleton (Basic)

If you don't want excess code you might not use, or prefer to build from an almost-empty foundation that covers the basic bootstrapping, then the skeleton schema is for you. The skeleton schema contains a lot of comments explaining why code is laid out in a certain way, and some other helpful tips/explanations. Make sure you give it a read!

You'll need to download the schema from [GitHub](#) into the folder name of your choice - just make sure it's all lowercase with no spaces. Our example for the sake of brevity will be `myschema`. Place the folder into `garrysmo/gamemodes`.

Next up is to modify the gamemode info so that Garry's Mod will properly recognize it. Rename `skeleton.txt` in your schema folder to your folder's name. In our example we would rename `skeleton.txt` to `myschema.txt`. Next, you'll need to modify the contents of `myschema.txt` and replace the existing information with your own - making sure to replace the `"skeleton"` at the top of the file to your folder's name. In our case we would replace it with `"myschema"`. Once you've renamed the file, you're all good to go!

Converting from Clockwork (Intermediate)

If you are looking to switch to Helix from Clockwork, you can follow the [conversion guide](#).

Starting from scratch (Intermediate)

You can always create the gamemode files yourself if you'd like (although we suggest the skeleton schema in general). In general, a schema is a gamemode that is derived from `helix` and automatically loads `schema/sh_schema.lua`. You shouldn't have your schema files outside of the `schema` folder. The files you'll need are as follows:

`gamemode/init.lua`

```
AddCSLuaFile("cl_init.lua")
DeriveGamemode("helix")
```

`gamemode/cl_init.lua`

```
DeriveGamemode("helix")
```

`schema/sh_schema.lua`

```
Schema.name = "My Schema"
Schema.author = "me!"
Schema.description = "My awesome schema."

-- include your other schema files
ix.util.Include("cl_schema.lua")
ix.util.Include("sv_schema.lua")
-- etc.
```


Class

<https://docs.gethelix.co/hooks/class/>

Class

Class setup hooks.

As with [Factions](#), [Classes](#) get their own hooks for when players leave/join a class, etc. These hooks are only valid in class tables that are created in `schema/classes/sh_classname.lua`, and cannot be used like regular gamemode hooks.

Functions

CLASS:CanSwitchTo(client)

Whether or not a player can switch to this class.

Parameters

- client [Player](#)
Client that wants to switch to this class

Returns

- bool
True if the player is allowed to switch to this class

Example Usage

```
function CLASS:CanSwitchTo(client)
    return client:IsAdmin() -- only admins allowed in this class!
end
```

CLASS:OnLeave(client)

Called when a character has left this class and has joined a different one. You can get the class the character has joined by calling `character:GetClass()`.

Parameters

- client [Player](#)
Player who left this class

CLASS:OnSet(client)

Called when a character has joined this class.

Parameters

- client [Player](#)

Player who has joined this class

Example Usage

```
function CLASS:OnSet(client)
    client:SetModel("models/police.mdl")
end
```

CLASS:OnSpawn(client)

Called when a character in this class has spawned in the world.

Parameters

- client [Player](#)

Player that has just spawned

Faction

<https://docs.gethelix.co/hooks/faction/>

Faction

Faction setup hooks.

Factions get their own hooks that are called for various reasons, but the most common one is to set up a character once it's created and assigned to a certain faction. For example, giving a police faction character a weapon on creation. These hooks are used in faction tables that are created in `schema/factions/sh_factionname.lua` and cannot be used like regular gamemode hooks.

Functions

FACTION:GetDefaultName(client)

Called when the default name for a character needs to be retrieved (i.e upon initial creation).

Parameters

- client [Player](#)

Client to get the default name for

Returns

- [string](#)

Default name for the newly created character

Example Usage

```
function FACTION:GetDefaultName(client)
    return "MPF-RCT." .. tostring(math.random(1, 99999))
end
```

FACTION:OnCharacterCreated(client, character)

Called when a character has been initially created and assigned to this faction.

Parameters

- client [Player](#)

Client that owns the character

- character [Character](#)

Character that has been created

Example Usage

```
function FACTION:OnCharacterCreated(client, character)
    local inventory = character:GetInventory()
    inventory:Add("pistol")
end
```

FACTION:OnSpawn(client)

Called when a character in this faction has spawned in the world.

Parameters

- client [Player](#)

Player that has just spawned

FACTION:OnTransferred(character)

Called when a player's character has been transferred to this faction.

Parameters

- character [Character](#)

Character that was transferred

Example Usage

```
function FACTION:OnTransferred(character)
    character:SetModel(self.models[1])
end
```

Plugin

<https://docs.gethelix.co/hooks/plugin/>

Plugin

Global hooks for general use.

Plugin hooks are regular hooks that can be used in your schema with `Schema:HookName(args)`, in your plugin with `PLUGIN:HookName(args)`, or in your addon with `hook.Add("HookName", function(args) end)`.

Functions

PLUGIN:AdjustCreationPayload(client, payload, newPayload)

Adjusts the data used just before creating a new character.

Parameters

- client [Player](#)
Player that is creating the character
- payload [table](#)
Table of data to be used for character creation
- newPayload [table](#)
Table of data be merged with the current payload

Example Usage

```
function PLUGIN:AdjustCreationPayload(client, payload, newPayload)
    newPayload.money = payload.attributes["stm"] -- Sets the characters initial money to the stamina
    attribute value.
end
```

PLUGIN:AdjustStaminaOffset(client, baseOffset)

Adjusts a player's current stamina offset amount. This is called when the player's stamina is about to be changed; every `0.25` seconds on the server, and every frame on the client.

Parameters

- client [Player](#)

Player whose stamina is changing

- baseOffset number

Amount the stamina is changing by. This can be a positive or negative number depending if they are exhausting or regaining stamina

Returns

- number

New offset to use

Example Usage

```
function PLUGIN:AdjustStaminaOffset(client, baseOffset)
    return baseOffset * 2 -- Drain/Regain stamina twice as fast.
end
```

Creates the business panel in the tab menu.

Returns

- bool

Whether or not to create the business menu

Example Usage

```
function PLUGIN:BuildBusinessMenu()
    return LocalPlayer():IsAdmin() -- Only builds the business menu for admins.
end
```

PLUGIN:CanAutoFormatMessage(speaker, chatType, text)

Whether or not a message can be auto formatted with punctuation and capitalization.

Parameters

- speaker [Player](#)

Player that sent the message

- chatType [string](#)

Chat type of the message. This will be something registered with [ix.chat.Register](#) - like `ic`, `ooc`, etc.

- text [string](#)

Unformatted text of the message

Returns

- bool

Whether or not to allow auto formatting on the message

Example Usage

```
function PLUGIN:CanAutoFormatMessage(speaker, chatType, text)
    return false -- Disable auto formatting outright.
end
```

PLUGIN:CanCreateCharacterInfo(suppress)

Whether or not certain information can be displayed in the character info panel in the tab menu.

Parameters

- suppress [table](#)

Information to **NOT** display in the UI - modify this to change the behaviour. This is a table of the names of some panels to avoid displaying. Valid names include:

- `time` - current in-game time
- `name` - name of the character
- `description` - description of the character
- `characterInfo` - entire panel showing a list of additional character info
- `faction` - faction name of the character
- `class` - name of the character's class if they're in one
- `money` - current money the character has
- `attributes` - attributes list for the character

Note that schemas/plugins can add additional character info panels.

Example Usage

```
function PLUGIN:CanCreateCharacterInfo(suppress)
    suppress.attributes = true -- Hides the attributes panel from the character info tab
end
```

PLUGIN:CanDrawAmmoHUD(weapon)

Whether or not the ammo HUD should be drawn.

Parameters

- weapon [Entity](#)

Weapon the player currently is holding

Returns

- bool

Whether or not to draw the ammo hud

Example Usage

```
function PLUGIN:CanDrawAmmoHUD(weapon)
    if (weapon:GetClass() == "weapon_frag") then -- Hides the ammo hud when holding grenades.
        return false
    end
end
```

PLUGIN:CanPlayerAccessDoor(client, door, access)

Called when a player tries to use abilities on the door, such as locking.

Parameters

- client [Player](#)

The client trying something on the door.

- door [Entity](#)

The door entity itself.

- access number

The access level used when called.

Returns

- bool

Whether or not to allow the client access.

Example Usage

```
function PLUGIN:CanPlayerAccessDoor(client, door, access)
    return true -- Always allow access.
end
```

PLUGIN:CanPlayerCombineItem(client, item, other)

Whether or not a player is allowed to combine an item `other` into the given `item`.

Parameters

- client [Player](#)

Player attempting to combine an item into another

- item number

instance ID of the item being dropped onto

- other number

instance ID of the item being combined into the first item, this can be invalid due to it being from clientside

Returns

- bool

Whether or not to allow the player to combine the items

Example Usage

```
function PLUGIN:CanPlayerCombineItem(client, item, other)
    local otherItem = ix.item.instances[other]

    if (otherItem and otherItem.uniqueID == "soda") then
        return false -- disallow combining any item that has a uniqueID equal to `soda`
    end
end
```

PLUGIN:CanPlayerCreateCharacter(client, payload)

Whether or not a player is allowed to create a new character with the given payload.

Parameters

- client [Player](#)

Player attempting to create a new character

- payload [table](#)

Data that is going to be used for creating the character

Returns

- bool

Whether or not the player is allowed to create the character. This function defaults to `true`, so you should only ever return `false` if you're disallowing creation. Otherwise, don't return anything as you'll prevent any other calls to this hook from running.

- [string](#)

Language phrase to use for the error message

- ...

Arguments to use for the language phrase

Example Usage

```
function PLUGIN:CanPlayerCreateCharacter(client, payload)
    if (!client:IsAdmin()) then
        return false, "notNow" -- only allow admins to create a character
    end
end
-- non-admins will see the message "You are not allowed to do this right now!"
```

PLUGIN:CanPlayerDropItem(client, item)

Whether or not a player is allowed to drop the given `item`.

Parameters

- client [Player](#)

Player attempting to drop an item

- item number

instance ID of the item being dropped

Returns

- bool

Whether or not to allow the player to drop the item

Example Usage

```
function PLUGIN:CanPlayerDropItem(client, item)
    return false -- Never allow dropping items.
end
```

PLUGIN:CanPlayerEarnSalary(client, faction)

Whether or not a player can earn money at regular intervals. This hook runs only if the player's character faction has a salary set - i.e `FACTION.pay` is set to something other than `0` for their faction.

Parameters

- client [Player](#)

Player to give money to

- faction [table](#)

Faction of the player's character

Returns

- bool

Whether or not to allow the player to earn salary

Example Usage

```
function PLUGIN:CanPlayerEarnSalary(client, faction)
    return client:IsAdmin() -- Restricts earning salary to admins only.
end
```

PLUGIN:CanPlayerEnterObserver(client)

Whether or not the player is allowed to enter observer mode. This is allowed only for admins by default and can be customized by server owners if the server is using a CAML-compliant admin mod.

Parameters

- client [Player](#)

Player attempting to enter observer

Returns

- bool

Whether or not to allow the player to enter observer

Example Usage

```
function PLUGIN:CanPlayerEnterObserver(client)
    return true -- Always allow observer.
end
```

PLUGIN:CanPlayerEquipItem(client, item)

Whether or not a player can equip the given `item`. This is called for items with `outfit`, `pacoutfit`, or `weapons` as their base. Schemas/plugins can utilize this hook for their items.

Parameters

- client [Player](#)

Player attempting to equip the item

- item [table](#)

Item being equipped

Returns

- bool

Whether or not to allow the player to equip the item

Example Usage

```
function PLUGIN:CanPlayerEquipItem(client, item)
    return client:IsAdmin() -- Restrict equipping items to admins only.
end
```

See Also

- [CanPlayerUnequipItem](#)

PLUGIN:CanPlayerHoldObject(client, entity)

Whether or not a player is allowed to hold an entity with the hands SWEP.

Parameters

- client [Player](#)

Player attempting to hold an entity

- entity [Entity](#)

Entity being held

Returns

- bool

Whether or not to allow the player to hold the entity

Example Usage

```
function PLUGIN:CanPlayerHoldObject(client, entity)
    return !(client:GetMoveType() == MOVETYPE_NOCLIP and !client:InVehicle()) -- Disallow players in
observer holding objects.
end
```

PLUGIN:CanPlayerInteractEntity(client, entity, option, data)

Whether or not a player is allowed to interact with an entity's interaction menu if it has one.

Parameters

- client [Player](#)

Player attempting interaction

- entity [Entity](#)

Entity being interacted with

- option [string](#)

Option selected by the player

- data

Any data passed with the interaction option

Returns

- bool

Whether or not to allow the player to interact with the entity

Example Usage

```
function PLUGIN:CanPlayerInteractEntity(client, entity, option, data)
  if (entity:GetClass() == "my_big_entity" and entity:GetPos():Distance(client:GetPos()) < 192) then
    return true -- Force allow interacting if within larger than default interact range of large entity
  end

  if (client:GetNetVar("drunk")) then
    return false -- Disallow interacting with an entity while drunk
  end
end
```

PLUGIN:CanPlayerInteractItem(client, action, item, data)

Whether or not a player is allowed to interact with an item via an inventory action (e.g picking up, dropping, transferring inventories, etc). Note that this is for an item table, not an item entity. This is called after [CanPlayerDropItem](#) and [CanPlayerTakeItem](#).

Parameters

- client [Player](#)

Player attempting interaction

- action [string](#)

The action being performed

- item

Item's instance ID or item table

- data

Any data passed with the action

Returns

- bool

Whether or not to allow the player to interact with the item

Example Usage

```
function PLUGIN:CanPlayerInteractItem(client, action, item, data)
    return false -- Disallow interacting with any item.
end
```

PLUGIN:CanPlayerJoinClass(client, class, info)

Whether or not a player is allowed to join a class.

Parameters

- client [Player](#)

Player attempting to join

- class number

ID of the class

- info [table](#)

The class table

Returns

- bool

Whether or not to allow the player to join the class

Example Usage

```
function PLUGIN:CanPlayerJoinClass(client, class, info)
    return client:IsAdmin() -- Restrict joining classes to admins only.
end
```

PLUGIN:CanPlayerKnock(client, entity)

Whether or not a player can knock on the door with the hands SWEP.

Parameters

- client [Player](#)

Player attempting to knock

- entity [Entity](#)

Door being knocked on

Returns

- bool

Whether or not to allow the player to knock on the door

Example Usage

```
function PLUGIN:CanPlayerKnock(client, entity)
    return false -- Disable knocking on doors outright.
end
```

PLUGIN:CanPlayerOpenShipment(client, entity)

Whether or not a player can open a shipment spawned from the business menu.

Parameters

- client [Player](#)

Player attempting to open the shipment

- entity [Entity](#)

Shipment entity

Returns

- bool

Whether or not to allow the player to open the shipment

Example Usage

```
function PLUGIN:CanPlayerOpenShipment(client, entity)
    return client:Team() == FACTION_BMD -- Restricts opening shipments to FACTION_BMD.
end
```

PLUGIN:CanPlayerSpawnContainer(client, model, entity)

Whether or not a player is allowed to spawn a container entity.

Parameters

- client [Player](#)
Player attempting to spawn a container
- model [string](#)
Model of the container being spawned
- entity [Entity](#)
Container entity

Returns

- bool
Whether or not to allow the player to spawn the container

Example Usage

```
function PLUGIN:CanPlayerSpawnContainer(client, model, entity)
    return client:IsAdmin() -- Restrict spawning containers to admins.
end
```

PLUGIN:CanPlayerTakeItem(client, item)

Whether or not a player is allowed to take an item and put it in their inventory.

Parameters

- client [Player](#)
Player attempting to take the item
- item [Entity](#)
Entity corresponding to the item

Returns

- bool
Whether or not to allow the player to take the item

Example Usage

```
function PLUGIN:CanPlayerTakeItem(client, item)
    return !(client:GetMoveType() == MOVETYPE_NOCLIP and !client:IsInVehicle()) -- Disallow players in
observer taking items.
end
```

PLUGIN:CanPlayerThrowPunch(client)

Whether or not the player is allowed to punch with the hands SWEP.

Parameters

- client [Player](#)

Player attempting throw a punch

Returns

- bool

Whether or not to allow the player to punch

Example Usage

```
function PLUGIN:CanPlayerThrowPunch(client)
    return client:GetCharacter():GetAttribute("str", 0) > 0 -- Only allow players with strength to punch.
end
```

PLUGIN:CanPlayerTradeWithVendor(client, entity, uniqueID, isSellingToVendor)

Whether or not a player can trade with a vendor.

Parameters

- client [Player](#)

Player attempting to trade

- entity [Entity](#)

Vendor entity

- uniqueID [string](#)

The uniqueID of the item being traded.

- isSellingToVendor boolean

If the client is selling to the vendor

Returns

- bool

Whether or not to allow the client to trade with the vendor

Example Usage

```
function PLUGIN:CanPlayerTradeWithVendor(client, entity, uniqueID, isSellingToVendor)
    return false -- Disallow trading with vendors outright.
end
```

PLUGIN:CanPlayerUnequipItem(client, item)

Whether or not a player can unequip an item.

Parameters

- client [Player](#)
Player attempting to unequip an item
- item [table](#)
Item being unequipped

Returns

- bool
Whether or not to allow the player to unequip the item

Example Usage

```
function PLUGIN:CanPlayerUnequipItem(client, item)
    return false -- Disallow unequipping items.
end
```

See Also

- [CanPlayerEquipItem](#)

PLUGIN:CanPlayerUseBusiness(client, uniqueID)

Whether or not a player can buy an item from the business menu.

Parameters

- client [Player](#)
Player that uses a business menu
- uniqueID [string](#)
The uniqueID of the business menu item

Returns

- bool

Whether or not to allow the player to buy an item from the business menu

Example Usage

```
function PLUGIN:CanPlayerUseBusiness(client, uniqueID)
    return false -- Disallow buying from the business menu.
end
```

PLUGIN:CanPlayerUseCharacter(client, character)

Whether or not a player can use a character.

Parameters

- client [Player](#)

Player that wants to use a character

- character [Character](#)

Character that a player wants to use

Returns

- bool

Whether or not to allow the player to load a character

Example Usage

```
function PLUGIN:CanPlayerUseCharacter(client, character)
    return false -- Disallow using any character.
end
```

PLUGIN:CanPlayerUseDoor(client, entity)

Whether or not a player can use a door.

Parameters

- client [Player](#)

Player that wants to use a door

- entity [Entity](#)

Door that a player wants to use

Returns

- bool

Whether or not to allow the player to use a door

Example Usage

```
function PLUGIN:CanPlayerUseDoor(client, character)
    return false -- Disallow using any door.
end
```

PLUGIN:CanPlayerUseVendor(activator, vendor)

Determines whether a player can use a vendor.

Parameters

- activator [Player](#)

The player attempting to use the vendor

- vendor [Entity](#)

The vendor entity being used

Returns

- bool

Returns false if the player can't use the vendor

PLUGIN:CanPlayerViewInventory()

Whether or not a player can view his inventory.

Returns

- bool

Whether or not to allow the player to view his inventory

Example Usage

```
function PLUGIN:CanPlayerViewInventory()
    return false -- Prevent player from viewing his inventory.
end
```

PLUGIN:CanSaveContainer(entity, inventory)

Whether or not to save a container.

Parameters

- entity [Entity](#)

Container entity to save

- inventory [table](#)

Container inventory

Returns

- bool

Whether or not to save a container

Example Usage

```
function PLUGIN:CanSaveContainer(entity, inventory)
    return false -- Disallow saving any container.
end
```

PLUGIN:CanTransferItem(item, currentInv, oldInv)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- item
- currentInv
- oldInv

PLUGIN:CharacterAttributeBoosted(client, character, attribID, boostID, boostAmount)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- character
- attribID
- boostID
- boostAmount

PLUGIN:CharacterAttributeUpdated(client, self, key, value)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- self
- key
- value

PLUGIN:CharacterDeleted(client, id, isCurrentChar)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- id
- isCurrentChar

PLUGIN:CharacterHasFlags(self, flags)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- self
- flags

PLUGIN:CharacterLoaded(character)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- character

PLUGIN:CharacterPostSave(character)

Called when a character was saved.

Parameters

- character [Character](#)
that was saved

PLUGIN:CharacterPreSave(character)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- character

PLUGIN:CharacterRecognized()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:CharacterRestored(character)

Called when a character was restored.

Parameters

- character [Character](#)
that was restored

PLUGIN:CharacterVarChanged(character, key, oldVar, value)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- character
- key
- oldVar
- value

PLUGIN:CharacterVendorTraded(client, entity, uniqueID, isSellingToVendor)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- entity
- uniqueID
- isSellingToVendor

PLUGIN:ChatboxCreated()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:ChatboxPositionChanged(x, y, width, height)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- x
- y
- width
- height

PLUGIN:ColorSchemeChanged(color)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- color

PLUGIN:ContainerRemoved(container, inventory)

Called when a container was removed.

Parameters

- container [Entity](#)
Container that was removed
- inventory [table](#)
Container inventory

PLUGIN:CreateCharacterInfo(panel)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- panel

PLUGIN:CreateCharacterInfoCategory(panel)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- panel

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- icon
- menu
- itemTable

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- tabs

PLUGIN:CreateShipment(client, entity)

Called when a shipment was created.

Parameters

- client [Player](#)

Player that ordered the shipment

- entity [Entity](#)

Shipment entity

PLUGIN:DatabaseConnected()

Called when a server has connected to the database.

PLUGIN:DatabaseConnectionFailed(error)

Called when a server failed to connect to the database.

Parameters

- error [string](#)

Error that prevented server from connecting to the database

PLUGIN:DoPluginIncludes(path, pluginTable)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- path
- pluginTable

PLUGIN:DrawCharacterOverview()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:DrawHelixModelView(panel, entity)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- panel
- entity

PLUGIN:DrawPlayerRagdoll(entity)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- entity

PLUGIN:GetCharacterDescription(client)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client

PLUGIN:GetCharacterName(speaker, chatType)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- speaker
- chatType

PLUGIN:GetChatPrefixInfo(text)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- text

PLUGIN:GetCrosshairAlpha(curAlpha)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- curAlpha

PLUGIN:GetDefaultAttributePoints(client, count)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- count

PLUGIN:GetDefaultCharacterName(client, faction)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- faction

PLUGIN:GetMaxPlayerCharacter(client)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client

PLUGIN:GetPlayerDeathSound(client)

Returns the sound to emit from the player upon death. If nothing is returned then it will use the default male/female death sounds.

Parameters

- client [Player](#)

Player that died

Returns

- [string](#)

Sound to play

OR

- bool

`false` if a sound shouldn't be played at all

Example Usage

```
function PLUGIN:GetPlayerDeathSound(client)
    -- play impact sound every time someone dies
    return "physics/body/body_medium_impact_hard1.wav"
end
```

```
function PLUGIN:GetPlayerDeathSound(client)
    -- don't play a sound at all
    return false
end
```

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- options

PLUGIN:GetPlayerIcon(speaker)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- speaker

PLUGIN:GetPlayerPainSound(client)

Returns the sound to emit from the player upon getting damage.

Parameters

- client [Player](#)

Client that received damage

Returns

- [string](#)

Sound to emit

Example Usage

```
function PLUGIN:GetPlayerPainSound(client)
    return "NPC_MetroPolice.Pain" -- Make players emit MetroPolice pain sound.
end
```

PLUGIN:GetPlayerPunchDamage(client, damage, context)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- damage
- context

PLUGIN:GetSalaryAmount(client, faction)

Returns the salary that character should get instead of his faction salary.

Parameters

- client [Player](#)

Client that is receiving salary

- faction [table](#)

Faction of the player's character

Returns

- number

Character salary

Example Usage

```
function PLUGIN:GetSalaryAmount(client, faction)
    return 0 -- Everyone get no salary.
end
```

See Also

- [CanPlayerEarnSalary](#)

PLUGIN:GetTypingIndicator(character, text)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- character
- text

PLUGIN:InitializedChatClasses()

Registers chat classes after the core framework chat classes have been registered. You should usually create your chat classes in this hook - especially if you want to reference the properties of a framework chat class.

Example Usage

```
function PLUGIN:InitializedChatClasses()
    -- let's say you wanted to reference an existing chat class's color
    ix.chat.Register("myclass", {
        format = "%s says \"%S\"",
        GetColor = function(self, speaker, text)
            -- make the chat class slightly brighter than the "ic" chat class
            local color = ix.chat.classes.ic:GetColor(speaker, text)

            return Color(color.r + 35, color.g + 35, color.b + 35)
        end,
        -- etc.
    })
end
```

```
    })  
end
```

See Also

- [ix.chat.Register](#)
- [ix.chat.classes](#)

PLUGIN:InitializedConfig()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:InitializedPlugins()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:InitializedSchema()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:InventoryItemAdded(oldInv, inventory, item)

Called when an item was added to the inventory.

Parameters

- oldInv [table](#)

Previous item inventory

- inventory [table](#)

New item inventory

- item [table](#)

Item that was added to the inventory

PLUGIN:InventoryItemRemoved(inventory, item)

Called when an item was removed from the inventory.

Parameters

- inventory [table](#)

Inventory from which item was removed

- item [table](#)

Item that was removed from the inventory

PLUGIN:IsCharacterRecognized(character, id)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- character
- id

PLUGIN:IsPlayerRecognized(client)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client

PLUGIN:IsRecognizedChatType(chatType)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- chatType

PLUGIN:LoadData()

Called when server is loading data.

PLUGIN:LoadFonts(font, genericFont)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- font
- genericFont

PLUGIN:LoadIntro()

Incomplete

Documentation for this section is incomplete and needs expanding.

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- subpanelName
- panel

PLUGIN:MessageReceived(client, info)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- info

PLUGIN:OnAreaChanged(oldID, newID)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- oldID
- newID

PLUGIN:OnCharacterCreated(client, character)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- character

PLUGIN:OnCharacterDisconnect(client, character)

Called when a player who uses a character has disconnected.

Parameters

- client [Player](#)
The player that has disconnected
- character [Character](#)
The character that the player was using

PLUGIN:OnCharacterFallover(client, entity, bFallenOver)

Called when a character was ragdolled or unragdolled.

Parameters

- client [Player](#)
Player that was ragdolled or unradolled
- entity [Entity](#)
Ragdoll that represents the player
- bFallenOver boolean
Whether or not the character was ragdolled or unragdolled

PLUGIN:OnCharacterGetup(client, ragdoll)

Called when a character has gotten up from the ground.

Parameters

- client [Player](#)

Player that has gotten up

- ragdoll [Entity](#)

Ragdoll used to represent the player

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- panel

PLUGIN:OnItemSpawned(entity)

Called whenever an item entity has spawned in the world. You can access the entity's item table with `entity:GetItemTable()`.

Parameters

- entity [Entity](#)

Spawned item entity

Example Usage

```
function PLUGIN:OnItemSpawned(entity)
    local item = entity:GetItemTable()
    -- do something with the item here
end
```

PLUGIN:OnItemTransferred(item, curInv, inventory)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- item
- curInv
- inventory

PLUGIN:OnLocalVarSet(key, var)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- key
- var

PLUGIN:OnPAC3PartTransferred(part)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- part

PLUGIN:OnPickupMoney(client, self)

Called when a player has picked up the money from the ground.

Parameters

- client [Player](#)
Player that picked up the money
- self [Entity](#)
Money entity

Returns

- bool
Whether or not to allow the player to pick up the money

Example Usage

```
function PLUGIN:OnPickupMoney(client, self)
    return false -- Disallow picking up money.
end
```

PLUGIN:OnPlayerAreaChanged(client, oldID, newID)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client

- oldID
- newID

PLUGIN:OnPlayerObserve(client, state)

Called when a player has entered or exited the observer mode.

Parameters

- client [Player](#)
Player that entered or exited the observer mode
- state boolean
Previous observer state

PLUGIN:OnPlayerOptionSelected(client, callingClient, option)

Called when a player has selected the entity interaction menu option while interacting with a player.

Parameters

- client [Player](#)
Player that other player has interacted with
- callingClient [Player](#)
Player that has interacted with with other player
- option [string](#)
Option that was selected

PLUGIN:OnPlayerPurchaseDoor(client, entity, bBuying, bCallOnDoorChild)

Called when a player has purchased or sold a door.

Parameters

- client [Player](#)

Player that has purchased or sold a door

- entity [Entity](#)

Door that was purchased or sold

- bBuying boolean

Whether or not the player is bying a door

- bCallOnDoorChild function

Function to call something on the door child

PLUGIN:OnPlayerRestricted(client)

Called when a player was restricted.

Parameters

- client [Player](#)

Player that was restricted

PLUGIN:OnPlayerUnRestricted(client)

Called when a player was unrestricted.

Parameters

- client [Player](#)

Player that was unrestricted

PLUGIN:OnSavedItemLoaded(loadedItems)

Called when a saved items were loaded.

Parameters

- loadedItems [table](#)

Table of items that were loaded

PLUGIN:OnWipeTables()

Called when server database are being wiped.

PLUGIN:PlayerEnterSequence(client, sequence, callback, time, bNoFreeze)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- sequence
- callback
- time
- bNoFreeze

PLUGIN:PlayerInteractEntity(client, entity, option, data)

Called when a player has interacted with an entity through the entity's interaction menu.

Parameters

- client [Player](#)
Player that performed interaction
- entity [Entity](#)
Entity being interacted with
- option [string](#)
Option selected by the player
- data
Any data passed with the interaction option

PLUGIN:PlayerInteractItem(client, action, item)

Called when a player has interacted with an item.

Parameters

- client [Player](#)

Player that interacted with an item

- action [string](#)

Action selected by the player

- item [table](#)

Item being interacted with

PLUGIN:PlayerJoinedClass(client, class, oldClass)

Called when a player has joined a class.

Parameters

- client [Player](#)

Player that has joined a class

- class number

Index of the class player has joined to

- oldClass number

Index of the player's previous class

PLUGIN:PlayerLeaveSequence(entity)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- entity

PLUGIN:PlayerLoadedCharacter(client, character, currentChar)

Called when a player has loaded a character.

Parameters

- client [Player](#)

Player that has loaded a character

- character [Character](#)

Character that was loaded

- currentChar [Character](#)

Character that player was using

PLUGIN:PlayerLockedDoor(client, door, partner)

Called when a player has locked a door.

Parameters

- client [Player](#)

Player that has locked a door

- door [Entity](#)

Door that was locked

- partner [Entity](#)

Door partner

PLUGIN:PlayerLockedVehicle(client, vehicle)

Called when a player has locked a vehicle.

Parameters

- client [Player](#)

Player that has locked a vehicle

- vehicle [Entity](#)

Vehicle that was locked

PLUGIN:PlayerMessageSend(speaker, chatType, text, anonymous, receivers, rawText)

Called when player has said something in the text chat.

Parameters

- speaker [Player](#)

Player that has said something in the text chat

- chatType [string](#)

Type of the chat that player used

- text [string](#)

Chat message that player send

- anonymous boolean

Whether or not message was anonymous

- receivers [table](#)

Players who will hear that message

- rawText [string](#)

Chat message without any formatting

Returns

- [string](#)

You can return text that will be shown instead

Example Usage

```
function PLUGIN:PlayerMessageSend(speaker, chatType, text, anonymous, receivers, rawText)
    return "Text" -- When a player writes something into chat, he will say "Text" instead.
end
```

PLUGIN:PlayerModelChanged(client, oldModel)

Called when a player model was changed.

Parameters

- client [Player](#)

Player whose model was changed

- oldModel [string](#)

Old player model

PLUGIN:PlayerStaminaGained(client)

Called when a player has got stamina.

Parameters

- client [Player](#)

Player who has got stamina

PLUGIN:PlayerStaminaLost(client)

Called when a player has lost stamina.

Parameters

- client [Player](#)

Player who has lost stamina

PLUGIN:PlayerThrowPunch(client, trace)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- trace

PLUGIN:PlayerUnlockedDoor(client, door, partner)

Called when a player has unlocked a door.

Parameters

- client [Player](#)

Player that has unlocked a door

- door [Entity](#)

Door that was unlocked

- partner [Entity](#)

Door partner

PLUGIN:PlayerUnlockedVehicle(client, vehicle)

Called when a player has unlocked a vehicle.

Parameters

- client [Player](#)

Player that has unlocked a vehicle

- vehicle [Entity](#)

Vehicle that was unlocked

PLUGIN:PlayerUse(client, entity)

Called when a player has used an entity.

Parameters

- client [Player](#)

Player who has used an entity

- entity [Entity](#)

Entity that was used by the player

PLUGIN:PlayerUseDoor(client, entity)

Called when a player has used a door.

Parameters

- client [Player](#)

Player who has used a door

- entity [Entity](#)

Door that was used by the player

PLUGIN:PlayerWeaponChanged(client, weapon)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- weapon

PLUGIN:PluginLoaded(uniqueID, pluginTable)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- uniqueID
- pluginTable

PLUGIN:PluginShouldLoad(uniqueID)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- uniqueID

PLUGIN:PluginUnloaded(uniqueID)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- uniqueID

PLUGIN:PopulateCharacterInfo(client, character, tooltip)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- character
- tooltip

PLUGIN:PopulateEntityInfo(entity, tooltip)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- entity
- tooltip

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- categories

PLUGIN:PopulateImportantCharacterInfo(entity, character, tooltip)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- entity
- character
- tooltip

PLUGIN:PopulateItemTooltip(tooltip, item)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- tooltip
- item

PLUGIN:PopulatePlayerTooltip(client, tooltip)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- tooltip

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- menu

PLUGIN:PostChatboxDraw(width, height, alpha)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- width
- height
- alpha

PLUGIN:PostDrawHelixModelView(panel, entity)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- panel
- entity

PLUGIN:PostDrawInventory(panel)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- panel

PLUGIN:PostLoadData()

Called when server data was loaded.

PLUGIN:PostPlayerLoadout(client)

Called after player loadout.

Parameters

- client [Player](#)

PLUGIN:PostPlayerSay(client, chatType, message, anonymous)

Called after player has said something in the text chat.

Parameters

- client [Player](#)
Player that has said something in the text chat
- chatType [string](#)
Type of the chat that player used
- message [string](#)
Chat message that player send
- anonymous boolean
Whether or not message was anonymous

PLUGIN:PostSetupActs()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:PreCharacterDeleted(client, character)

Called before character deletion.

Parameters

- client [Player](#)
Character owner
- character [Character](#)
Character that will be deleted

PLUGIN:PrePlayerLoadedCharacter(client, character, currentChar)

Called before character loading.

Parameters

- client [Player](#)
Player that loading a character
- character [Character](#)
Character that will be loaded
- currentChar [Character](#)
Character that player is using

PLUGIN:PrePlayerMessageSend(client, chatType, message, bAnonymous)

Called before a message sent by a player is processed to be sent to other players - i.e this is ran as early as possible and before things like the auto chat formatting. Can be used to prevent the message from being sent at all.

Parameters

- client [Player](#)
Player sending the message
- chatType [string](#)
Chat class of the message
- message [string](#)
Contents of the message
- bAnonymous boolean
Whether or not the player is sending the message anonymously

Returns

- bool
Whether or not to prevent the message from being sent

Example Usage

```
function PLUGIN:PrePlayerMessageSend(client, chatType, message, bAnonymous)
    if (!client:IsAdmin()) then
        return false -- only allow admins to talk in chat
    end
end
```

PLUGIN:SaveData()

Called when server is saving data.

PLUGIN:ScreenResolutionChanged(width, height)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- width
- height

PLUGIN:SetupActs()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:SetupAreaProperties()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:ShipmentItemTaken(client, uniqueID, amount)

Called when a player has taken a shipment item.

Parameters

- client [Player](#)
Player that has taken a shipment item
- uniqueID [string](#)

UniqueID of the shipment item that was taken

- amount number

Amount of the items that were taken

PLUGIN:ShouldBarDraw(bar)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- bar

PLUGIN:ShouldDeleteSavedItems()

Whether or not the server should delete saved items.

Returns

- bool

Whether or not the server should delete saved items

Example Usage

```
function PLUGIN:ShouldDeleteSavedItems()  
    return true -- Delete all saved items.  
end
```

PLUGIN:ShouldDisplayArea(newID)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- newID

PLUGIN:ShouldDrawCrosshair(client, weapon)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client
- weapon

PLUGIN:ShouldDrawItemSize(item)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- item

PLUGIN:ShouldHideBars()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:ShouldPermakillCharacter(client, character, inflictor, attacker)

Whether or not a character should be permakilled upon death. This is only called if the `permakill` server config is enabled.

Parameters

- client [Player](#)
Player to permakill
- character [Character](#)
Player's current character
- inflictor [Entity](#)
Entity that inflicted the killing blow
- attacker [Entity](#)
Other player or entity that killed the player

Returns

- bool
`false` if the player should not be permakilled

Example Usage

```
function PLUGIN:ShouldPermakillCharacter(client, character, inflictor, attacker)
    if (client:IsAdmin()) then
        return false -- all non-admin players will have their character permakilled
    end
end
```

PLUGIN:ShouldPlayerDrowned(client)

Whether or not player should drown.

Parameters

- client [Player](#)

Player that is underwater

Returns

- bool

Whether or not player should drown

Example Usage

```
function PLUGIN:ShouldPlayerDrowned(client)
    return false -- Players will not drown.
end
```

PLUGIN:ShouldRemoveRagdollOnDeath(client)

Whether or not remove player ragdoll on death.

Parameters

- client [Player](#)

Player that died

Returns

- bool

Whether or not remove player ragdoll on death

Example Usage

```
function PLUGIN:ShouldRemoveRagdollOnDeath(client)
    return false -- Player ragdolls will not be removed.
end
```

PLUGIN:ShouldRestoreInventory(characterID, inventoryID, inventoryType)

Whether or not to restore character inventory.

Parameters

- characterID number
ID of the character
- inventoryID number
ID of the inventory
- inventoryType [string](#)
Type of the inventory

Returns

- bool
Whether or not to restore character inventory

Example Usage

```
function PLUGIN:ShouldRestoreInventory(characterID, inventoryID, inventoryType)
    return false -- Character inventories will not be restored.
end
```

PLUGIN:ShouldShowPlayerOnScoreboard(client)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- client

PLUGIN:ShouldSpawnClientRagdoll(client)

Whether or not spawn player ragdoll on death.

Parameters

- client [Player](#)
Player that died

Returns

- bool

Whether or not spawn player ragdoll on death

Example Usage

```
function PLUGIN:ShouldSpawnClientRagdoll(client)
    return false -- Player ragdolls will not be spawned.
end
```

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- entity

PLUGIN:ThirdPersonToggled(oldValue, value)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- oldValue
- value

PLUGIN:UpdateCharacterInfo(panel, character)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- panel
- character

PLUGIN:UpdateCharacterInfoCategory(panel, character)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- panel
- character

PLUGIN:VoiceDistanceChanged(newValue)

Called when the distance on which the voice can be heard was changed.

Parameters

- newValue number
New voice distance

PLUGIN:WeaponCycleSound()

Incomplete

Documentation for this section is incomplete and needs expanding.

PLUGIN:WeaponSelectSound(weapon)

Incomplete

Documentation for this section is incomplete and needs expanding.

Parameters

- weapon

ix

Top-level library containing all Helix libraries.

A large majority of the framework is split into respective libraries that reside within [ix](#).

Tables

type

[View source »](#)

A table of variable types that are used throughout the framework. It represents types as a table with the keys being the name of the type, and the values being some number value. **You should never directly use these number values!** Using the values from this table will ensure backwards compatibility if the values in this table change.

This table also contains the numerical values of the types as keys. This means that if you need to check if a type exists, or if you need to get the name of a type, you can do a table lookup with a numerical value. Note that special types are not included since they are not real types that can be compared with.

Fields

- string

A regular string. In the case of [ix.command.Add](#), this represents one word.

- text

A regular string. In the case of [ix.command.Add](#), this represents all words concatenated into a string.

- number

Any number.

- player

Any player that matches the given query string in [ix.util.FindPlayer](#).

- steamid

A string that matches the Steam ID format of `STEAM_X:X:XXXXXXXX`.

- character

Any player's character that matches the given query string in [ix.util.FindPlayer](#).

- bool

A string representation of a bool - `false` and `0` will return `false`, anything else will return `true`.

- color

A color represented by its red/green/blue/alpha values.

- vector

A 3D vector represented by its x/y/z values.

- optional

This is a special type that can be bitwise OR'd with any other type to make it optional. Currently only supported in [ix.command.Add](#).

- array

This is a special type that can be bitwise OR'd with any other type to make it an array of that type. Currently only supported in [ix.option.Add](#).

Example Usage

```
-- checking if type exists
print(ix.type[2] != nil)
> true

-- getting name of type
print(ix.type[ix.type.string])
> "string"
```

See Also

- [ix.command.Add](#)
- [ix.option.Add](#)

ix.act

Provides players the ability to perform animations.

Functions

ix.act.Register(name, modelClass, data)

[View source »](#)

Registers a sequence as a performable animation.

Parameters

- name [string](#)
Name of the animation (in CamelCase)
- modelClass [string](#)
Model class to add this animation to
- data [table](#)
An `ActInfoStructure` table describing the animation

ix.act.Remove(name)

[View source »](#)

Removes a sequence from being performable if it has been previously registered.

Parameters

- name [string](#)
Name of the animation

ix.anim

Player model animation.

Helix comes with support for using NPC animations/models as regular player models by manually translating animations. There are a few standard animation sets that are built-in that should cover most non-player models:

```
citizen_male  
citizen_female  
metrocop  
overwatch  
vortigaunt  
player  
zombie  
fastZombie
```

If you find that your models are T-posing when they work elsewhere, you'll probably need to set the model class for your model with [ix.anim.SetModelClass](#) in order for the correct animations to be used. If you'd like to add your own animation class, simply add to the [ix.anim](#) table with a model class name and the required animation translation table.

Functions

ix.anim.GetModelClass(model)

[View source »](#)

Gets a model's animation class.

Parameters

- model [string](#)

Model to get the animation class for

Returns

- [string](#)

Animation class of the model

OR

- nil

If there was no animation associated with the given model

Example Usage

```
ix.anim.GetModelClass("models/police.mdl")  
> metrocop
```

ix.anim.SetModelClass(model, class)

[View source »](#)

Sets a model's animation class.

Parameters

- model [string](#)
Model name to set the animation class for
- class [string](#)
Animation class to assign to the model

Example Usage

```
ix.anim.SetModelClass("models/police.mdl", "metrocop")
```

ix.char

Character creation and management.

NOTE: For the most part you shouldn't use this library unless you know what you're doing. You can very easily corrupt character data using these functions!

Functions

ix.char.Create(data, callback)

[View source »](#)

Creates a character object with its assigned properties and saves it to the database.

Parameters

- data [table](#)
Properties to assign to this character. If fields are missing from the table, then it will use the default value for that property
- callback function
Function to call after the character saves

ix.char.New(data, id, client, steamID)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Creates a new empty [Character](#) object. If you are looking to create a usable character, see [ix.char.Create](#).

Parameters

- data [table](#)
Character vars to assign
- id number

Unique ID of the character

- client [Player](#)

Player that will own the character

- steamID [string](#) default: client:SteamID64()

SteamID64 of the player that will own the character

ix.char.Restore(client, callback, bNoCache, id)

[View source »](#)

Loads all of a player's characters into memory.

Parameters

- client [Player](#)

Player to load the characters for

- callback function default: nil

Function to call when the characters have been loaded

- bNoCache boolean default: false

Whether or not to skip the cache; players that leave and join again later will already have their characters loaded which will skip the database query and load quicker

- id number default: nil

The ID of a specific character to load instead of all of the player's characters

ix.char.RestoreVars(data, characterInfo)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Adds character properties to a table. This is done automatically by [ix.char.Restore](#), so that should be used instead if you are loading characters.

Parameters

- data [table](#)

Table of fields to apply to the table. If this is an SQL query object, it will instead populate the query with `SELECT` statements for each applicable character var in [ix.char.vars](#).

- characterInfo [table](#)

Table to apply the properties to. This can be left as `nil` if an SQL query object is passed in [data](#)

Tables

ix.char.cache

[View source »](#)

Functions similar to [ix.char.loaded](#), but is serverside only. This contains a table of all loaded characters grouped by the SteamID64 of the player that owns them.

ix.char.loaded

[View source »](#)

Characters that are currently loaded into memory. This is **not** a table of characters that players are currently using. Characters are automatically loaded when a player joins the server. Entries are not cleared once the player disconnects, as some data is needed after the player has disconnected. Clients will also keep their own version of this table, so don't expect it to be the same as the server's.

The keys in this table are the IDs of characters, and the values are the [Character](#) objects that the ID corresponds to.

Example Usage

```
print(ix.char.loaded[1])  
> character[1]
```

ix.char.vars

[View source »](#)

Variables that are stored on characters. This table is populated automatically by `ix.char.RegisterVar`.

Example Usage

```
print(ix.char.vars["name"])  
> table: 0xdeadbeef
```


ix.chat

Chat manipulation and helper functions.

Chat messages are a core part of the framework - it's takes up a good chunk of the gameplay, and is also used to interact with the framework. Chat messages can have types or "classes" that describe how the message should be interpreted. All chat messages will have some type of class: `ic` for regular in-character speech, `me` for actions, `ooc` for out-of-character, etc. These chat classes can affect how the message is displayed in each player's chatbox. See [ix.chat.Register](#) and [ChatClassStructure](#) to create your own chat classes.

Functions

ix.chat.Format(text)

[View source »](#)

Formats a string to fix basic grammar - removing extra spacing at the beginning and end, capitalizing the first character, and making sure it ends in punctuation.

Parameters

- text [string](#)
String to format

Returns

- [string](#)
Formatted string

Example Usage

```
print(ix.chat.Format("hello"))  
> Hello.
```

```
print(ix.chat.Format("wow!"))  
> Wow!
```

ix.chat.Parse(client, message, bNoSend)

[View source »](#)

Identifies which chat mode should be used.

Parameters

- client [Player](#)
Player who is speaking
- message [string](#)
Message to parse
- bNoSend boolean default: false
Whether or not to send the chat message after parsing

Returns

- [string](#)
Name of the chat type
- [string](#)
Message that was parsed
- bool
Whether or not the speaker should be anonymous

ix.chat.Register(chatType, data)

[View source »](#)

Registers a new chat type with the information provided. Chat classes should usually be created inside of the `InitializedChatClasses` hook.

Parameters

- chatType [string](#)
Name of the chat type
- data [ChatClassStructure](#)
Properties and functions to assign to this chat class

Example Usage

```
-- this is the "me" chat class taken straight from the framework as an example
ix.chat.Register("me", {
    format = "** %s %s",
    color = Color(255, 50, 50),
    CanHear = ix.config.Get("chatRange", 280) * 2,
    prefix = {"/Me", "/Action"},
    description = "@cmdMe",
    indicator = "chatPerforming",
```

```
        deadCanChat = true
    })
```

See Also

- [ChatClassStructure](#)

ix.chat.Send(speaker, chatType, text, bAnonymous, receivers, data)

[View source »](#)

Send a chat message using the specified chat type.

Parameters

- speaker [Player](#)
Player who is speaking
- chatType [string](#)
Name of the chat type
- text [string](#)
Message to send
- bAnonymous boolean default: false
Whether or not the speaker should be anonymous
- receivers [table](#) default: nil
The players to replicate send the message to
- data [table](#) default: nil
Additional data for this chat message

Tables

ChatClassStructure

[View source »](#)

Chat messages can have different classes or "types" of messages that have different properties. This can include how the text is formatted, color, hearing distance, etc.

Fields

- prefix [string](#)

What the player must type before their message in order to use this chat class. For example, having a prefix of `/Y` will require to type `/Y I am yelling` in order to send a message with this chat class. This can also be a table of strings if you want to allow multiple prefixes, such as `{"//", "/00C"}`.

NOTE: the prefix should usually start with a `/` to be consistent with the rest of the framework. However, you are able to use something different like the `L00C` chat class where the prefixes are `.//`, `[[`, and `/L00C`.

- `noSpaceAfter` bool default: `false`

Whether or not the `prefix` can be used without a space after it. For example, the `00C` chat class allows you to type `//my message` instead of `// my message`. **NOTE:** this only works if the last character in the prefix is non-alphanumeric (i.e. `noSpaceAfter` with `/Y` will not work, but `/!` will).

- `description` [string](#) optional

Description to show to the user in the chatbox when they're using this chat class

- `format` [string](#) default: `"%s: \"%s\""`

How to format a message with this chat class. The first `%s` will be the speaking player's name, and the second one will be their message

- `color` [Color](#) default: `Color(242 230 160)`

Color to use when displaying a message with this chat class

- `indicator` [string](#) default: `"chatTyping"`

Language phrase to use when displaying the typing indicator above the speaking player's head

- `bNoIndicator` bool default: `false`

Whether or not to avoid showing the typing indicator above the speaking player's head

- `font` [string](#) default: `ixChatFont`

Font to use for displaying a message with this chat class

- `deadCanChat` bool default: `false`

Whether or not a dead player can send a message with this chat class

- `CanHear` number

This can be either a `number` representing how far away another player can hear this message. IC messages will use the `chatRange` config, for example. This can also be a function, which returns `true` if the given listener can hear the message emitted from a speaker.

```
-- message can be heard by any player 1000 units away from the speaking player
CanHear = 1000
```

OR

```
CanHear = function(self, speaker, listener)
  -- the speaking player will be heard by everyone
  return true
end
```

- CanSay function optional

Function to run to check whether or not a player can send a message with this chat class. By default, it will return `false` if the player is dead and `deadCanChat` is `false`. Overriding this function will prevent `deadCanChat` from working, and you must implement this functionality manually.

```
CanSay = function(self, speaker, text)
  -- the speaker will never be able to send a message with this chat class
  return false
end
```

- GetColor function optional

Function to run to set the color of a message with this chat class. You should generally stick to using `color`, but this is useful for when you want the color of the message to change with some criteria.

```
GetColor = function(self, speaker, text)
  -- each message with this chat class will be colored a random shade of red
  return Color(math.random(120, 200), 0, 0)
end
```

- OnChatAdd function optional

Function to run when a message with this chat class should be added to the chatbox. If using this function, make sure you end the function by calling `chat.AddText` in order for the text to show up.

NOTE: using your own `OnChatAdd` function will prevent `color`, `GetColor`, or `format` from being used since you'll be overriding the base function that uses those properties. In such cases you'll need to add that functionality back in manually. In general, you should avoid overriding this function where possible. The `data` argument in the function is whatever is passed into the same `data` argument in `ix.chat.Send`.

```
OnChatAdd = function(self, speaker, text, bAnonymous, data)
  -- adds white text in the form of "Player Name: Message contents"
  chat.AddText(color_white, speaker:GetName(), ": ", text)
end
```

See Also

- [ix.chat.Register](#)

ix.chat.classes

[View source »](#)

List of all chat classes that have been registered by the framework, where each key is the name of the chat class, and value is the chat class data. Accessing a chat class's data is useful for when you want to copy some functionality or properties to use in your own. Note that if you're accessing this table, you should do so inside of the `InitializedChatClasses` hook.

Example Usage

```
print(ix.chat.classes.ic.format)  
> "%s says \"%s\""
```

ix.class

Helper library for loading/getting class information.

Classes are temporary assignments for characters - analogous to a "job" in a faction. For example, you may have a police faction in your schema, and have "police recruit" and "police chief" as different classes in your faction. Anyone can join a class in their faction by default, but you can restrict this as you need with `CLASS.CanSwitchTo`.

Functions

ix.class.CanSwitchTo(client, class)

[View source »](#)

Determines if a player is allowed to join a specific class.

Parameters

- client [Player](#)
Player to check
- class number
Index of the class

Returns

- bool
Whether or not the player can switch to the class.
- [string](#)
The reason why the player cannot switch (if applicable).

Example Usage

```
-- Check if a player can join class ID 2.
-- For our example, they can't- because they are in the wrong faction.
local canJoin, reason = ix.class.CanSwitchTo(player, 2)
if (!canJoin) then
    print("Player cannot join class: "..reason)
end
> Player cannot join class: not correct team
```

ix.class.Get(identifier)

[View source »](#)

Gets a class definition table by numeric identifier.

Parameters

- identifier number

Numeric class identifier.

Returns

- [table](#) or nil

Class definition table (see [class](#) table docs).

Example Usage

```
-- Print the name of class ID 1
print(ix.class.Get(1).name)
> Citizen
```

ix.class.GetPlayers(class)

[View source »](#)

Retrieves all players currently assigned to a specific class.

Parameters

- class number

Index of the class

Returns

- [table](#)

Numerically indexed table of players in the class, or an empty table if none are found.

Example Usage

```
-- Print all players in class ID 1
for _, ply in ipairs(ix.class.GetPlayers(1)) do
    print(ply:GetName())
end
> Player1
> Player2
> etc
```


ix.class.LoadFromDir(directory)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Loads classes from a directory.

Parameters

- directory [string](#)

The path to the class files.

Fields

class

[View source »](#)

Class definition table returned by [ix.class.Get](#).

- class [table](#)

Class definition table. Default keys:

- `index` (number) - numeric class ID
- `uniqueID` (string) - stable identifier (e.g. `citizen`)
- `name` (string)
- `description` (string)
- `faction` (number) - Faction ID this class belongs to
- `isDefault` (boolean) - whether it is the default class for the faction
- `limit` (number) - maximum number of players allowed in this class (0 for unlimited)
- `CanSwitchTo` (function|nil) - optional callback to check if a player may switch

ix.command

Registration, parsing, and handling of commands.

Commands can be ran through the chat with slash commands or they can be executed through the console. Commands can be manually restricted to certain usergroups using a [CAMI](#)-compliant admin mod.

Functions

ix.command.Add(command, data)

[View source »](#)

Creates a new command.

Parameters

- command [string](#)
Name of the command (recommended in UpperCamelCase)
- data [CommandStructure](#)
Data describing the command

See Also

- [CommandStructure](#)
- [CommandArgumentsStructure](#)

ix.command.ExtractArgs(text)

[View source »](#)

Returns a table of arguments from a given string. Words separated by spaces will be considered one argument. To have an argument containing multiple words, they must be contained within quotation marks.

Parameters

- text [string](#)
String to extract arguments from

Returns

- [table](#)

Arguments extracted from string

Example Usage

```
PrintTable(ix.command.ExtractArgs("these are \"some arguments\""))
> 1 = these
> 2 = are
> 3 = some arguments
```

ix.command.FindAll(identifier, bSorted, bReorganize, bRemoveDupes)

[View source »](#)

Returns an array of potential commands by unique id. When bSorted is true, the commands will be sorted by name. When bReorganize is true, it will move any exact match to the top of the array. When bRemoveDupes is true, it will remove any commands that have the same NAME.

Parameters

- identifier [string](#)

Search query

- bSorted boolean default: false

Whether or not to sort the commands by name

- bReorganize boolean default: false

Whether or not any exact match will be moved to the top of the array

- bRemoveDupes boolean default: false

Whether or not to remove any commands that have the same name

Returns

- [table](#)

Array of command tables whose name partially or completely matches the search query

ix.command.FindPlayer(client, name)

[View source »](#)

Attempts to find a player by an identifier. If unsuccessful, a notice will be displayed to the specified player. The search criteria is derived from [ix.util.FindPlayer](#).

Parameters

- client [Player](#)

Player to give a notification to if the player could not be found

- name [string](#)

Search query

Returns

- player

Player that matches the given search query

OR

- nil

If a player could not be found

See Also

- [ix.util.FindPlayer](#)

ix.command.HasAccess(client, command)

[View source »](#)

Returns true if a player is allowed to run a certain command.

Parameters

- client [Player](#)

Player to check access for

- command [string](#)

Name of the command to check access for

Returns

- bool

Whether or not the player is allowed to run the command

ix.command.Parse(client, text, realCommand, arguments)

[View source »](#)

Parses a chat string and runs the command if one is found. Specifically, it checks for commands in a string with the format `/CommandName some arguments`

Parameters

- client [Player](#)

Player who is executing the command

- text [string](#)

Input string to search for the command format

- realCommand [string](#) optional

Specific command to check for. If this is specified, it will not try to run any command that's found at the beginning - only if it matches `realCommand`

- arguments [table](#) optional

Array of arguments to pass to the command. If not specified, it will try to extract it from the string specified in `text` using [ix.command.ExtractArgs](#)

Returns

- bool

Whether or not a command has been found

Example Usage

```
ix.command.Parse(player.GetByID(1), "/roll 10")
```

ix.command.Run(client, command, arguments)

[View source »](#)

Forces a player to execute a command by name.

Parameters

- client [Player](#)

Player who is executing the command

- command [string](#)

Full name of the command to be executed. This string gets lowered, but it's good practice to stick with the exact name of the command

- arguments [table](#)

Array of arguments to be passed to the command

Example Usage

```
ix.command.Run(player.GetByID(1), "Roll", {10})
```

ix.command.Send(command, ...)

[View source »](#)

Request the server to run a command. This mimics similar functionality to the client typing `/CommandName` in the chatbox.

Parameters

- command `string`
Unique ID of the command
- ...
Arguments to pass to the command

Example Usage

```
ix.command.Send("roll", 10)
```

Tables

CommandArgumentsStructure

[View source »](#)

Rather than checking the validity for arguments in your command's `OnRun` function, you can have Helix do it for you to reduce the amount of boilerplate code that needs to be written. This can be done by populating the `arguments` field.

When using the `arguments` field in your command, you are specifying specific types that you expect to receive when the command is ran successfully. This means that before `OnRun` is called, the arguments passed to the command from a user will be verified to be valid. Each argument is an `ix.type` entry that specifies the expected type for that argument. Optional arguments can be specified by using a bitwise OR with the special `ix.type.optional` type. When specified as optional, the argument can be `nil` if the user has not entered anything for that argument - otherwise it will be valid.

Note that optional arguments must always be at the end of a list of arguments - or rather, they must not follow a required argument. The `syntax` field will be automatically populated when using strict arguments, which means you shouldn't fill out the `syntax` field yourself. The arguments you specify will have the same names as the arguments in your `OnRun` function.

Consider this example command:

```
ix.command.Add("CharSlap", {
    description = "Slaps a character with a large trout.",
    adminOnly = true,
    arguments = {
        ix.type.character,
        bit.bor(ix.type.number, ix.type.optional)
    }
})
```

```

},
OnRun = function(self, client, target, damage)
    -- WHAM!
end
})

```

Here, we've specified the first argument called `target` to be of type `character`, and the second argument called `damage` to be of type `number`. The `damage` argument is optional, meaning that the command will still run if the user has not specified any value for the damage. In this case, we'll need to check if it was specified by doing a simple `if (damage) then`. The syntax field will be automatically populated with the value `"<target: character> [damage: number]"`.

CommandStructure

[View source »](#)

When registering commands with `ix.command.Add`, you'll need to pass in a valid command structure. This is simply a table with various fields defined to describe the functionality of the command.

Fields

- OnRun function

This function is called when the command has passed all the checks and can execute. The first two arguments will be the running command table and the calling player. If the arguments field has been specified, the arguments will be passed as regular function parameters rather than in a table.

When the arguments field is defined: `OnRun(self, client, target, length, message)`

When the arguments field is NOT defined: `OnRun(self, client, arguments)`

- description [string](#) default: `"@noDesc"`

The help text that appears when the user types in the command. If the string is prefixed with `"@"`, it will use a language phrase.

- argumentNames [table](#) default: `nil`

An array of strings corresponding to each argument of the command. This ignores the name that's specified in the `OnRun` function arguments and allows you to use any string to change the text that displays in the command's syntax help. When using this field, make sure that the amount is equal to the amount of arguments, as such:

```

COMMAND.arguments = {ix.type.character, ix.type.number}
COMMAND.argumentNames = {"target char", "cash (1-1000)"}

```

- arguments [table](#) optional

If this field is defined, then additional checks will be performed to ensure that the arguments given to the command are valid. This removes extra boilerplate code since all the passed arguments are guaranteed to be valid. See [CommandArgumentsStructure](#) for more information.

- `adminOnly` boolean default: `false`

Provides an additional check to see if the user is an admin before running.

- `superAdminOnly` boolean default: `false`

Provides an additional check to see if the user is a superadmin before running.

- `privilege` [string](#) default: `nil`

Manually specify a privilege name for this command. It will always be prefixed with `"Helix - "`. This is used in the case that you want to group commands under the same privilege, or use a privilege that you've already defined (i.e grouping `/CharBan` and `/CharUnban` into the `Helix - Ban Character` privilege).

- `OnCheckAccess` function default: `nil`

This callback checks whether or not the player is allowed to run the command. This callback should **NOT** be used in conjunction with `adminOnly` or `superAdminOnly`, as populating those fields create a custom `OnCheckAccess` callback for you internally. This is used in cases where you want more fine-grained access control for your command.

Keep in mind that this is a **SHARED** callback; the command will not show up the client if the callback returns `false`.

ix.config

Helper library for creating/setting config options.

Functions

ix.config.Add(key, value, description, callback, data, bNoNetworking, bSchemaOnly)

[View source »](#)

Creates a config option with the given information.

Parameters

- key [string](#)
Unique ID of the config
- value
Default value that this config will have
- description [string](#)
Description of the config
- callback function default: nil
Function to call when config is changed
- data [table](#) default: nil
Additional settings for this config option
- bNoNetworking boolean default: false
Whether or not to prevent networking the config
- bSchemaOnly boolean default: false
Whether or not the config is for the schema only

ix.config.Get(key, default)

[View source »](#)

Retrieves a value of a config option. If it is not set, it'll return the default that you've specified.

Parameters

- key [string](#)
Unique ID of the config
- default
Default value to return if the config is not set

Returns

- any
Value associated with the key, or the default that was given if it doesn't exist

ix.config.Load()

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Loads all saved config options from disk.

ix.config.Save()

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Saves all config options to disk.

ix.config.Set(key, value)

[View source »](#)

Sets the value of a config option.

Parameters

- key [string](#)
Unique ID of the config
- value

New value to assign to the config

ix.config.SetDefault(key, value)

[View source »](#)

Sets the default value for a config option.

Parameters

- key [string](#)

Unique ID of the config

- value

Default value for the config option

ix.currency

A library representing the server's currency system.

Functions

ix.currency.Get(amount)

[View source »](#)

Returns a formatted string according to the current currency.

Parameters

- amount number

The amount of cash being formatted.

Returns

- [string](#)

The formatted string.

ix.currency.Set(symbol, singular, plural, model)

[View source »](#)

Sets the currency type.

Parameters

- symbol [string](#)

The symbol of the currency.

- singular [string](#)

The name of the currency in it's singular form.

- plural [string](#)

The name of the currency in it's plural form.

- model [string](#)

The model of the currency entity.

ix.currency.Spawn(pos, amount, angle)

[View source »](#)

Spawns an amount of cash at a specific location on the map.

Parameters

- pos vector
The position of the money to be spawned.
- amount number
The amount of cash being spawned.
- angle angle default: angle_zero
The angle of the entity being spawned.

Returns

- entity
The spawned money entity.

ix.data

Helper library for reading/writing files to the data folder.

Functions

ix.data.Delete(key, bGlobal, bIgnoreMap)

[View source »](#)

Deletes the contents of a saved file in the `data/helix` folder.

Parameters

- key `string`
Name of the file to delete
- bGlobal boolean default: false
Whether or not the data is in the `data/helix` folder, or the `data/helix/schema` folder, where `schema` is the name of the current schema.
- bIgnoreMap boolean default: false
Whether or not to ignore the map and delete from the schema folder, rather than `data/helix/schema/map`, where `map` is the name of the current map.

Returns

- bool
Whether or not the deletion has succeeded

ix.data.Get(key, default, bGlobal, bIgnoreMap, bRefresh)

[View source »](#)

Retrieves the contents of a saved file in the `data/helix` folder.

Parameters

- key `string`

Name of the file to load

- default

Value to return if the file could not be loaded successfully

- bGlobal boolean default: false

Whether or not the data is in the `data/helix` folder, or the `data/helix/schema` folder, where `schema` is the name of the current schema.

- bIgnoreMap boolean default: false

Whether or not to ignore the map and load from the schema folder, rather than `data/helix/schema/map`, where `map` is the name of the current map.

- bRefresh boolean default: false

Whether or not to skip the cache and forcefully load from disk.

Returns

- any

Value associated with the key, or the default that was given if it doesn't exist

ix.data.Set(key, value, bGlobal, bIgnoreMap)

[View source »](#)

Populates a file in the `data/helix` folder with some serialized data.

Parameters

- key [string](#)

Name of the file to save

- value

Some sort of data to save

- bGlobal boolean default: false

Whether or not to write directly to the `data/helix` folder, or the `data/helix/schema` folder, where `schema` is the name of the current schema.

- bIgnoreMap boolean default: false

Whether or not to ignore the map and save in the schema folder, rather than `data/helix/schema/map`, where `map` is the name of the current map.

ix.date

Persistent date and time handling.

All of Lua's time functions are dependent on the Unix epoch, which means we can't have dates that go further than 1970. This library remedies this problem. Time/date is represented by a [date](#) object that is queried, instead of relying on the seconds since the epoch.

Futher documentation

This library makes use of a third-party date library found at <https://github.com/Tieske/date> - you can find all documentation regarding the [date](#) object and its methods there.

Functions

ix.date.Construct(currentDate)

[View source »](#)

Returns a date object from a table or serialized date.

Parameters

- currentDate
Date to construct

Returns

- date
Constructed date object

ix.date.Get()

[View source »](#)

Returns the currently set date.

Returns

- date
Current in-game date

ix.date.GetFormatted(format, currentDate)

[View source »](#)

Returns a string formatted version of a date.

Parameters

- format [string](#)
Format string
- currentDate date default: nil
Date to format. If nil, it will use the currently set date

Returns

- [string](#)
Formatted date

ix.date.GetSerialized(currentDate)

[View source »](#)

Returns a serialized version of a date. This is useful when you need to network a date to clients, or save a date to disk.

Parameters

- currentDate date default: nil
Date to serialize. If nil, it will use the currently set date

Returns

- [table](#)
Serialized date

ix.date.Initialize()

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Loads the date from disk.

ix.date.ResolveOffset()

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Updates the internal in-game date/time representation and resets the offset.

ix.date.Save()

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Saves the current in-game date to disk.

ix.date.Send(client)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Sends the current date to a player. This is done automatically when the player joins the server.

Parameters

- client [Player](#) default: nil

Player to send the date to, or `nil` to send to everyone

ix.date.UpdateTimescale(secondsPerMinute)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Updates the time scale of the in-game date/time. The time scale is given in seconds per minute (i.e how many real life seconds it takes for an in-game minute to pass). You should avoid using this function and use the in-game config menu to change the time scale instead.

Parameters

- secondsPerMinute number

New time scale

ix.faction

Helper library for loading/getting faction information.

Functions

ix.faction.Get(identifier)

[View source »](#)

Retrieves a faction table.

Parameters

- identifier
Index or name of the faction

Returns

- [table](#)
Faction table

Example Usage

```
print(ix.faction.Get(Entity(1):Team()).name)  
> "Citizen"
```

ix.faction.GetIndex(uniqueID)

[View source »](#)

Retrieves a faction index.

Parameters

- uniqueID [string](#)
Unique ID of the faction

Returns

- number
Faction index

ix.faction.HasWhitelist(faction)

[View source »](#)

Returns true if a faction requires a whitelist.

Parameters

- faction number
Index of the faction

Returns

- bool
Whether or not the faction requires a whitelist

ix.faction.LoadFromDir(directory)

[View source »](#)

Loads factions from a directory.

Parameters

- directory [string](#)
The path to the factions files.

ix.flag

Grants abilities to characters.

Flags are a simple way of adding/removing certain abilities to players on a per-character basis. Helix comes with a few flags by default, for example to restrict spawning of props, usage of the physgun, etc. All flags will be listed in the `Flags` section of the `Help` menu. Flags are usually used when server validation is required to allow a player to do something on their character. However, it's usually preferable to use in-character methods over flags when possible (i.e restricting the business menu to characters that have a permit item, rather than using flags to determine availability).

Flags are a single alphanumeric character that can be checked on the server. Serverside callbacks can be used to provide functionality whenever the flag is added or removed. For example:

```
ix.flag.Add("z", "Access to some cool stuff.", function(client, bGiven)
    print("z flag given:", bGiven)
end)

Entity(1):GetCharacter():GiveFlags("z")
> z flag given: true

Entity(1):GetCharacter():TakeFlags("z")
> z flag given: false

print(Entity(1):GetCharacter():HasFlags("z"))
> false
```

Check out [Character:GiveFlags](#) and [Character:TakeFlags](#) for additional info.

Functions

ix.flag.Add(flag, description, callback)

[View source »](#)

Creates a flag. This should be called shared in order for the client to be aware of the flag's existence.

Parameters

- flag [string](#)
Alphanumeric character to use for the flag
- description [string](#)

Description of the flag

- callback function

Function to call when the flag is given or taken from a player

ix.inventory

Inventory manipulation and helper functions.

Functions

ix.inventory.Get(invID)

[View source »](#)

Retrieves an inventory table.

Parameters

- invID number
Index of the inventory

Returns

- [Inventory](#)
Inventory table

ix.inventory.Restore(invID, width, height, callback)

[View source »](#)

Loads an inventory and associated items from the database into memory. If you are passing a table into `invID`, it requires a table where the key is the inventory ID, and the value is a table of the width and height values. See below for an example.

Parameters

- invID
Inventory ID or table of inventory IDs
- width number
Width of inventory (this is not used when passing a table to `invID`)
- height number
Height of inventory (this is not used when passing a table to `invID`)
- callback function

Function to call when inventory is restored

Example Usage

```
ix.inventory.Restore({  
    [10] = {5, 5},  
    [11] = {7, 4}  
})  
-- inventories 10 and 11 with sizes (5, 5) and (7, 4) will be loaded
```

ix.item

Item manipulation and helper functions.

Functions

ix.item.Get(identifier)

[View source »](#)

Retrieves an item table.

Parameters

- identifier [string](#)
Unique ID of the item

Returns

- item
Item table

Example Usage

```
print(ix.item.Get("example"))  
> "item[example][0]"
```

ix.item.Spawn(uniqueID, position, callback, angles, data)

[View source »](#)

Instances and spawns a given item type.

Parameters

- uniqueID [string](#)
Unique ID of the item
- position vector

The position in which the item's entity will be spawned

- callback function default: nil

Function to call when the item entity is created

- angles angle default: angle_zero

The angles at which the item's entity will spawn

- data [table](#) default: nil

Additional data for this item instance

ix.lang

Multi-language phrase support.

Helix has support for multiple languages, and you can easily leverage this system for use in your own schema, plugins, etc. Languages will be loaded from the schema and any plugins in `languages/sh_<language>.lua`, where `<language>` is the id of a language (`english` for English, `french` for French, etc). The structure of a language file is a table of phrases with the key as its phrase ID and the value as its translation for that language. For example, in `plugins/area/languages/sh_english.lua`:

```
LANGUAGE = {  
    area = "Area",  
    areas = "Areas",  
    areaEditMode = "Area Edit Mode",  
    -- etc.  
}
```

The phrases defined in these language files can be used with the `L` global function:

```
print(L("areaEditMode"))  
> Area Edit Mode
```

All phrases are formatted with [string.format](#), so if you wish to add some info in a phrase you can use standard Lua string formatting arguments:

```
print(L("areaDeleteConfirm", "Test"))  
> Are you sure you want to delete the area "Test"?
```

Phrases are also usable on the server, but only when trying to localize a phrase based on a client's preferences. The server does not have a set language. An example:

```
Entity(1):ChatPrint(L("areaEditMode"))  
> -- "Area Edit Mode" will print in the player's chatbox
```

Functions

ix.lang.AddTable(language, data)

[View source »](#)

Adds phrases to a language. This is used when you aren't adding entries through the files in the `languages/` folder. A common use case is adding language phrases in a single-file plugin.

Parameters

- language [string](#)

The ID of the language

- data [table](#)

Language data to add to the given language

Example Usage

```
ix.lang.AddTable("english", {  
    myPhrase = "My Phrase"  
})
```

ix.lang.LoadFromDir(directory)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Loads language files from a directory.

Parameters

- directory [string](#)

Directory to load language files from

ix.log

Logging helper functions.

Predefined flags:

```
FLAG_NORMAL
FLAG_SUCCESS
FLAG_WARNING
FLAG_DANGER
FLAG_SERVER
FLAG_DEV
```

Functions

ix.log.Add(client, logType, ...)

[View source »](#)

Adds a typed log entry. The entry is visible to admins with log access, printed to the server console, and written through the logging system.

Parameters

- client [Player](#) or nil
Player tied to the log (can be nil).
- logType [string](#)
Type registered with [ix.log.AddType](#).
- ...
Arguments for the formatter.

Example Usage

```
ix.log.Add(client, "charMoneyGive", giverName, receiverName, amount)
```

See Also

- [ix.log.AddType](#)
- [ix.log.AddRaw](#)

ix.log.AddRaw(logString, bNoSave)

[View source »](#)

Adds a log entry without using a log type. Sends the message to admins with log access, prints it to the server console, and writes it through log handlers unless `bNoSave` is true.

Parameters

- `logString` [string](#)
Pre-formatted log message.
- `bNoSave` boolean optional
If true, skips handler output (e.g. file logging).

Example Usage

```
ix.log.AddRaw("Server is restarting in 5 minutes.")
```

See Also

- [ix.log.Add](#)

ix.log.AddType(logType, format, flag)

[View source »](#)

Registers a log type for [ix.log.Add](#). `format` is either a format string or a function `(client, ...) -> string`.

`flag` is a display/severity tag (see `FLAG_*`). If omitted, it behaves like a normal log.

Parameters

- `logType` [string](#)
Type name used when calling [ix.log.Add](#).
- `format` [string](#) or function
Format string or formatter function.
- `flag` number optional
One of the `FLAG_*` constants.

Example Usage

```
ix.log.AddType("charMoneyGive", "%s gave %s %d tokens.", FLAG_SUCCESS)
```

ix.log.CallHandler(event, ...)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Calls a specific event on all registered log handlers.

Parameters

- event
- ...

ix.log.Parse(client, logType, ...)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Turns a log type + args into (message, flag) . Missing type falls back to a warning string. No formatter returns -1 (caller bails).

Parameters

- client [Player](#) or nil

Player tied to the log.

- logType [string](#)

Log type name.

- ...

Arguments for the formatter.

Returns

- [string](#) or number

Message string, or -1 to skip logging.

- number

Flag from the registered type.

ix.log.Send(receivers, logString, flag)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Sends a log entry to clients.

Parameters

- receivers [Player](#) or [table](#)

Target player(s).

- logString [string](#)

Log message.

- flag number optional

`FLAG_*` value (defaults to normal).

ix.menu

Entity menu manipulation.

The [menu](#) library allows you to open up a context menu of arbitrary options whose callbacks will be ran when they are selected from the panel that shows up for the player.

Functions

[View source »](#)

Checks whether or not an entity menu is currently open.

Returns

- boolean

Whether or not an entity menu is open

[View source »](#)

Notifies the server of an option that was chosen for the given entity.

Parameters

- entity [Entity](#)

Entity to call option on

- choice [string](#)

Option that was chosen

- data

Extra data to send to the entity

[View source »](#)

Opens up a context menu for the given entity.

Parameters

- options [MenuOptionsStructure](#)

Data describing what options to display

- entity [Entity](#) optional

Entity to send commands to

Returns

- boolean

Whether or not the menu opened successfully. It will fail when there is already a menu open.

Tables

[View source »](#)

You'll need to pass a table of options to [ix.menu.Open](#) to populate the menu. This table consists of strings as its keys and functions as its values. These correspond to the text displayed in the menu and the callback to run, respectively.

Example usage:

```
ix.menu.Open({
  Drink = function()
    print("Drink option selected!")
  end,
  Take = function()
    print("Take option selected!")
  end
}, ents.GetByIndex(1))
```

This opens a menu with the options "Drink" and "Take" which will print a message when you click on either of the options.

ix.notice

Notification helper functions

Functions

ix.util.Notify(message, recipient)

[View source »](#)

Sends a notification to a specified recipient.

Parameters

- message [string](#)
Message to notify
- recipient [Player](#) default: nil
Player to be notified

ix.util.NotifyLocalized(message, recipient, ...)

[View source »](#)

Sends a translated notification to a specified recipient.

Parameters

- message [string](#)
Message to notify
- recipient [Player](#) default: nil
Player to be notified
- ...
Arguments to pass to the translated message

ix.option

Client-side configuration management.

The `option` library provides a cleaner way to manage any arbitrary data on the client without the hassle of managing CVars. It is analagous to the `ix.config` library, but it only deals with data that needs to be stored on the client.

To get started, you'll need to define an option in a client realm so the framework can be aware of its existence. This can be done in the `cl_init.lua` file of your schema, or in an `if (CLIENT) then` statement in the `sh_plugin.lua` file of your plugin:

```
ix.option.Add("headbob", ix.type.bool, true)
```

If you need to get the value of an option on the server, you'll need to specify `true` for the `bNetworked` argument in `ix.option.Add`. NOTE: You also need to define your option in a shared realm, since the server now also needs to be aware of its existence. This makes it so that the client will send that option's value to the server whenever it changes, which then means that the server can now retrieve the value that the client has the option set to. For example, if you need to get what language a client is using, you can simply do the following:

```
ix.option.Get(player.GetByID(1), "language", "english")
```

This will return the language of the player, or `"english"` if one isn't found. Note that `"language"` is a networked option that is already defined in the framework, so it will always be available. All options will show up in the options menu on the client, unless `hidden` returns `true` when using `ix.option.Add`.

Note that the labels for each option in the menu will use a language phrase to show the name. For example, if your option is named `headbob`, then you'll need to define a language phrase called `optHeadbob` that will be used as the option title.

Functions

ix.option.Add(key, optionType, default, data)

[View source »](#)

Creates a client-side configuration option with the given information.

Parameters

- key `string`

Unique ID for this option

- optionType [ix.type](#)

Type of this option

- default

Default value that this option will have - this can be nil if needed

- data [OptionStructure](#)

Additional settings for this option

Example Usage

```
ix.option.Add("animationScale", ix.type.number, 1, {  
    category = "appearance",  
    min = 0.3,  
    max = 2,  
    decimals = 1  
})
```

ix.option.Get(client, key, default)

[View source »](#)

Retrieves an option value from the specified player. If it is not set, it'll return the default that you've specified. This function will error when an invalid player is passed.

Parameters

- client [Player](#)

Player to retrieve option value from

- key [string](#)

Unique ID of the option

- default

Default value to return if the option is not set

Returns

- any

Value associated with the key

OR

- any

The given default if the option is not set

ix.option.Get(key, default)

[View source »](#)

Retrieves an option value for the local player. If it is not set, it'll return the default that you've specified.

Parameters

- key [string](#)
Unique ID of the option
- default
Default value to return if the option is not set

Returns

- any
Value associated with the key
- OR
- any
The given default if the option is not set

ix.option.GetAll()

[View source »](#)

Returns all of the available options. Note that this does contain the actual values of the options, just their properties.

Returns

- [table](#)
Table of all options

Example Usage

```
PrintTable(ix.option.GetAll())
> language:
>   bNetworked = true
>   default = english
>   type = 512
-- etc.
```

ix.option.GetAllByCategories(bRemoveHidden)

[View source »](#)

Returns all of the available options grouped by their categories. The returned table contains category tables, that contain all the options in that category as an array (this is so you can sort them if you'd like).

Parameters

- `bRemoveHidden` boolean default: false

Remove entries that are marked as hidden

Returns

- [table](#)

Table of all options

Example Usage

```
PrintTable(ix.option.GetAllByCategories())
> general:
>     1:
>         key = language
>         bNetworked = true
>         default = english
>         type = 512
-- etc.
```

ix.option.Load()

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Loads all saved options from disk.

ix.option.Save()

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Saves all options to disk.

ix.option.Set(key, value, bNoSave)

[View source »](#)

Sets an option value for the local player. This function will error when an invalid key is passed.

Parameters

- key [string](#)
Unique ID of the option
- value
New value to assign to the option
- bNoSave boolean default: false
Whether or not to avoid saving

ix.option.SetDefault(key, value)

[View source »](#)

Sets the default value for a user option.

Parameters

- key [string](#)
Unique ID of the option
- value
Default value for the user option

ix.option.Sync()

[View source »](#)

Syncs all networked options to the server.

Tables

OptionStructure

[View source »](#)

You can specify additional optional arguments for [ix.option.Add](#) by passing in a table of specific fields as the fourth argument.

Fields

- phrase [string](#) default: "opt" .. key

The phrase to use when displaying in the UI. The default value is your option key in UpperCamelCase, prefixed with "opt". For example, if your key is "exampleOption", the default phrase will be "optExampleOption".

- description [string](#) default: "optd" .. key

The phrase to use in the tooltip when hovered in the UI. The default value is your option key in UpperCamelCase, prefixed with "optd". For example, if your key is "exampleOption", the default phrase will be "optdExampleOption".

- category [string](#) default: "misc"

The category that this option should reside in. This is purely for aesthetic reasons when displaying the options in the options menu. When displayed in the UI, it will take the form of L("category name"). This means that you must create a language phrase for the category name - otherwise it will only show as the exact string you've specified. If no category is set, it will default to "misc".

- min number default: 0

The minimum allowed amount when setting this option. This field is not applicable to any type other than `ix.type.number`.

- max number default: 10

The maximum allowed amount when setting this option. This field is not applicable to any type other than `ix.type.number`.

- decimals number default: 0

How many decimals to constrain to when using a number type. This field is not applicable to any type other than `ix.type.number`.

- bNetworked boolean default: false

Whether or not the server should be aware of this option for each client.

- OnChanged function optional

The function to run when this option is changed - this includes whether it was set by the player, or through code using [ix.option.Set](#).

```
OnChanged = function(oldValue, value)
    print("new value is", value)
end
```

- hidden function optional

The function to check whether the option should be hidden from the options menu.

- populate function optional

The function to run when the option needs to be added to the menu. This is a required field for any array options. It should return a table of entries where the key is the value to set in [ix.option.Set](#), and the value is the display name for the entry. An example:

```
populate = function()
  return {
    ["english"] = "English",
    ["french"] = "French",
    ["spanish"] = "Spanish"
  }
end
```

ix.storage

Player manipulation of inventories.

This library provides an easy way for players to manipulate other inventories. The only functions that you should need are [ix.storage.Open](#) and [ix.storage.Close](#). When opening an inventory as a storage item, it will display both the given inventory and the player's inventory in the player's UI, which allows them to drag items to and from the given inventory.

Example usage:

```
ix.storage.Open(client, inventory, {
    name = "Filing Cabinet",
    entity = ents.GetByIndex(3),
    bMultipleUsers = true,
    searchText = "Rummaging...",
    searchTime = 4
})
```

Functions

ix.storage.AddReceiver(client, inventory, bDontSync)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Adds a receiver to a given inventory with a storage context.

Parameters

- client [Player](#)

Player to sync storage for

- inventory [Inventory](#)

Inventory to sync storage for

- bDontSync boolean

Whether or not to skip syncing the storage to the client. If this is `true`, the storage panel will not show up for the player

ix.storage.Close(inventory)

[View source »](#)

Forcefully makes clients close this inventory if they have it open.

Parameters

- inventory [Inventory](#)

Inventory to close

ix.storage.CreateContext(inventory, info)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Creates a storage context on the given inventory.

Parameters

- inventory [Inventory](#)

Inventory to create a storage context for

- info [table](#)

Information to store on the context

ix.storage.InUse(inventory)

[View source »](#)

Returns whether or not the given inventory has a storage context and is being looked at by other players.

Parameters

- inventory [Inventory](#)

Inventory to check

Returns

- bool

Whether or not [inventory](#) is in use

ix.storage.InUseBy(inventory, client)

[View source »](#)

Returns whether or not an inventory is in use by a specific player.

Parameters

- inventory [Inventory](#)

Inventory to check

- client [Player](#)

Player to check

Returns

- bool

Whether or not the player is using the given [inventory](#)

ix.storage.Open(client, inventory, info)

[View source »](#)

Makes a player open an inventory that they can interact with. This can be called multiple times on the same inventory, if the info passed allows for multiple users.

Parameters

- client [Player](#)

Player to open the inventory for

- inventory [Inventory](#)

Inventory to open

- info [table](#)

[StorageInfoStructure](#) describing the storage properties

ix.storage.RemoveContext(inventory)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Removes a storage context from an inventory if it exists.

Parameters

- inventory [Inventory](#)

Inventory to remove a storage context from

ix.storage.RemoveReceiver(client, inventory, bDontRemove)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Removes a storage receiver and removes the context if there are no more receivers.

Parameters

- client [Player](#)

Player to remove from receivers

- inventory [Inventory](#)

Inventory with storage context to remove receiver from

- bDontRemove boolean

Whether or not to skip removing the storage context if there are no more receivers

ix.storage.Sync(client, inventory)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Synchronizes an inventory with a storage context to the given client.

Parameters

- client [Player](#)

Player to sync storage for

- inventory [Inventory](#)

Inventory to sync storage for

Tables

StorageInfoStructure

[View source »](#)

There are some parameters you can customize when opening an inventory as a storage object with `ix.storage.Open`.

Fields

- entity entity

Entity to "attach" the inventory to. This is used to provide a location for the inventory for things like making sure the player doesn't move too far away from the inventory, etc. This can also be a `player` object.

- id number default: inventory id

The ID of the nventory. This defaults to the inventory passed into `ix.Storage.Open`.

- name [string](#) default: "Storage"

Title to display in the UI when the inventory is open.

- bMultipleUsers boolean default: false

Whether or not multiple players are allowed to view this inventory at the same time.

- searchTime number default: 0

How long the player has to wait before the inventory is opened.

- text [string](#) default: "@storageSearching"

Text to display to the user while opening the inventory. If prefixed with `"@"`, it will display a language phrase.

- OnPlayerClose function optional

Called when a player who was accessing the inventory has closed it. The argument passed to the callback is the player who closed it.

- data [table](#) default: {}

Table of arbitrary data to send to the client when the inventory has been opened.

ix.util

Various useful helper functions.

Functions

ix.util.DimColor(color, multiplier, alpha)

[View source »](#)

Returns a dimmed version of the given color by the given scale.

Parameters

- color color
Color to dim
- multiplier number
What to multiply the red, green, and blue values by
- alpha number default: 255
Alpha to use in dimmed color

Returns

- color
Dimmed color

Example Usage

```
print(ix.util.DimColor(Color(100, 100, 100, 255), 0.5))  
> 50 50 50 255
```

ix.util.DrawBlur(panel, amount, passes, alpha)

[View source »](#)

Blurs the content underneath the given panel. This will fall back to a simple darkened rectangle if the player has blurring disabled.

Parameters

- panel panel

Panel to draw the blur for

- amount number default: 5

Intensity of the blur. This should be kept between 0 and 10 for performance reasons

- passes number default: 0.2

Quality of the blur. This should be kept as default

- alpha number default: 255

Opacity of the blur

Example Usage

```
function PANEL:Paint(width, height)
    ix.util.DrawBlur(self)
end
```

ix.util.DrawBlurAt(x, y, width, height, amount, passes, alpha)

[View source »](#)

Draws a blurred rectangle with the given position and bounds. This shouldn't be used for panels, see [ix.util.DrawBlur](#) instead.

Parameters

- x number

X-position of the rectangle

- y number

Y-position of the rectangle

- width number

Width of the rectangle

- height number

Height of the rectangle

- amount number default: 5

Intensity of the blur. This should be kept between 0 and 10 for performance reasons

- passes number default: 0.2

Quality of the blur. This should be kept as default

- alpha number default: 255

Opacity of the blur

Example Usage

```
hook.Add("HUDPaint", "MyHUDPaint", function()  
    ix.util.DrawBlurAt(0, 0, ScrW(), ScrH())  
end)
```

ix.util.DrawText(text, x, y, color, alignX, alignY, font, alpha)

[View source »](#)

Draws some text with a shadow.

Parameters

- text [string](#)
Text to draw
- x number
X-position of the text
- y number
Y-position of the text
- color [color](#)
Color of the text to draw
- alignX number default: TEXT_ALIGN_LEFT
Horizontal alignment of the text, using one of the `TEXT_ALIGN_*` constants
- alignY number default: TEXT_ALIGN_LEFT
Vertical alignment of the text, using one of the `TEXT_ALIGN_*` constants
- font [string](#) default: "ixGenericFont"
Font to use for the text
- alpha number default: color.a * 0.575
Alpha of the shadow

ix.util.EmitQueuedSounds(entity, sounds, delay, spacing, volume, pitch)

[View source »](#)

Emits sounds one after the other from an entity.

Parameters

- entity [Entity](#)
Entity to play sounds from
- sounds [table](#)
Sound paths to play
- delay number
[opt=0] How long to wait before starting to play the sounds
- spacing number
[opt=0.1] How long to wait between playing each sound
- volume number
[opt=75] The sound level of each sound
- pitch number
[opt=100] Pitch percentage of each sound

Returns

- number
How long the entire sequence of sounds will take to play

ix.util.ExpandCamelCase(input, bNoUpperFirst)

[View source »](#)

Returns a string that is the given input with spaces in between each CamelCase word. This function will ignore any words that do not begin with a capital letter. The words `ooc`, `looc`, `afk`, and `url` will be automatically transformed into uppercase text. This will not capitalize non-ASCII letters due to limitations with Lua's pattern matching.

Parameters

- input [string](#)
String to expand
- bNoUpperFirst boolean default: false

Whether or not to avoid capitalizing the first character. This is useful for lowerCamelCase

Returns

- [string](#)

Expanded CamelCase string

Example Usage

```
print(ix.util.ExpandCamelCase("HelloWorld"))  
> Hello World
```

ix.util.FindPlayer(identifier, bAllowPatterns)

[View source »](#)

Attempts to find a player by matching their name or Steam ID.

Parameters

- identifier [string](#)
Search query
- bAllowPatterns boolean default: false
Whether or not to accept Lua patterns in `identifier`

Returns

- player
Player that matches the given search query - this will be `nil` if a player could not be found

ix.util.FormatStringNamed(format, Arguments)

[View source »](#)

Returns a string that has the named arguments in the format string replaced with the given arguments.

Parameters

- format [string](#)
Format string
- Arguments table or ...
to pass to the formatted string. If passed a table, it will use that table as the lookup table for the named arguments. If passed multiple arguments, it will replace the arguments in the string in order.

Example Usage

```
print(ix.util.FormatStringNamed("Hi, my name is {name}.", {name = "Bobby"}))  
> Hi, my name is Bobby.
```

```
print(ix.util.FormatStringNamed("Hi, my name is {name}.", "Bobby"))  
> Hi, my name is Bobby.
```

ix.util.GetCharacters()

[View source »](#)

Returns an iterator for characters. The resulting key/values will be a player and their corresponding characters. This iterator skips over any players that do not have a valid character loaded.

Returns

- Iterator

Example Usage

```
for client, character in ix.util.GetCharacters() do  
    print(client, character)  
end  
> Player [1][Bot01]    character[1]  
> Player [2][Bot02]    character[2]  
-- etc.
```

ix.util.GetMaterial(materialPath)

[View source »](#)

Returns a cached copy of the given material, or creates and caches one if it doesn't exist. This is a quick helper function if you aren't locally storing a `Material()` call.

Parameters

- materialPath [string](#)
Path to the material

Returns

- material
The cached material
OR
- nil
If the material doesn't exist in the filesystem

ix.util.GetStringTime(text)

[View source »](#)

Gets the amount of seconds from a given formatted string. If no time units are specified, it is assumed minutes. The valid values are as follows:

- `s` - Seconds
- `m` - Minutes
- `h` - Hours
- `d` - Days
- `w` - Weeks
- `mo` - Months
- `y` - Years

Parameters

- text [string](#)

Text to interpret a length of time from

Returns

- number

Amount of seconds from the length interpreted from the given string

OR

- 0

If the given string does not have a valid time

Example Usage

```
print(ix.util.GetStringTime("5y2d7w"))
> 162086400 -- 5 years, 2 days, 7 weeks
```

ix.util.GetTypeFromValue(value)

[View source »](#)

Returns the [ix.type](#) of the given value.

Parameters

- value

Value to get the type of

Returns

- [ix.type](#)

Type of value

Example Usage

```
print(ix.util.GetTypeFromValue("hello"))  
> 2 -- i.e the value of ix.type.string
```

See Also

- [ix.type](#)

ix.util.GetUTCTime()

[View source »](#)

Gets the current time in the UTC time-zone.

Returns

- number

Current time in UTC

ix.util.Include(fileName, realm)

[View source »](#)

Includes a lua file based on the prefix of the file. This will automatically call `include` and `AddCSLuaFile` based on the current realm. This function should always be called shared to ensure that the client will receive the file from the server.

Parameters

- fileName [string](#)

Path of the Lua file to include. The path is relative to the file that is currently running this function

- realm [string](#) optional

Realm that this file should be included in. You should usually ignore this since it will be automatically be chosen based on the `SERVER` and `CLIENT` globals. This value should either be `"server"` or `"client"` if it is filled in manually

ix.util.IncludeDir(directory, bFromLua)

[View source »](#)

Includes multiple files in a directory.

Parameters

- directory [string](#)

Directory to include files from

- bFromLua boolean optional

Whether or not to search from the base `lua/` folder, instead of contextually basing from `schema/` or `gamemode/`

See Also

- [ix.util.Include](#)

ix.util.IsColor(input)

[View source »](#)

Returns `true` if the given input is a color table. This is necessary since the engine [IsColor](#) function only checks for color metatables - which are not used for regular Lua color types.

Parameters

- input

Input to check

Returns

- bool

Whether or not the input is a color

ix.util.MetatableSafeTableMerge(destination, source)

[View source »](#)

Merges the contents of the second table with the content in the first one. The destination table will be modified. If element is table but not metatable object, value's elements will be changed only.

Parameters

- destination [table](#)

The table you want the source table to merge with

- source [table](#)

The table you want to merge with the destination table

Returns

- any

table

ix.util.PushBlur(drawFunc)

[View source »](#)

Pushes a 3D2D blur to be rendered in the world. The draw function will be called next frame in the `PostDrawOpaqueRenderables` hook.

Parameters

- drawFunc function

Function to call when it needs to be drawn

ix.util.ResetStencilValues()

[View source »](#)

Resets all stencil values to known good (i.e defaults)

ix.util.SanitizeType(type, input)

[View source »](#)

Sanitizes an input value with the given type. This function ensures that a valid type is always returned. If a valid value could not be found, it will return the default value for the type. This only works for simple types - e.g it does not work for player, character, or Steam ID types.

Parameters

- type [ix.type](#)

Type to check for

- input

Value to sanitize

Returns

- any

Sanitized value

Example Usage

```
print(ix.util.SanitizeType(ix.type.number, "123"))
> 123
print(ix.util.SanitizeType(ix.type.bool, 1))
> true
```

See Also

- [ix.type](#)

ix.util.StringMatches(a, b)

[View source »](#)

Checks to see if two strings are equivalent using a fuzzy manner. Both strings will be lowered, and will return `true` if the strings are identical, or if `b` is a substring of `a`.

Parameters

- `a` [string](#)

First string to check

- `b` [string](#)

Second string to check

Returns

- `bool`

Whether or not the strings are equivalent

ix.util.StripRealmPrefix(name)

[View source »](#)

Removes the realm prefix from a file name. The returned string will be unchanged if there is no prefix found.

Parameters

- `name` [string](#)

String to strip prefix from

Returns

- [string](#)

String stripped of prefix

Example Usage

```
print(ix.util.StripRealmPrefix("sv_init.lua"))
> init.lua
```

ix.util.WrapText(text, maxWidth, font)

[View source »](#)

Wraps text so it does not pass a certain width. This function will try and break lines between words if it can, otherwise it will break a word if it's too long.

Parameters

- text [string](#)
Text to wrap
- maxWidth number
Maximum allowed width in pixels
- font [string](#) default: "ixChatFont"
Font to use for the text

Classes

Character

<https://docs.gethelix.co/classes/character/>

Character

Contains information about a player's current game state.

Characters are a fundamental object type in Helix. They are distinct from players, where players are the representation of a person's existence in the server that owns a character, and their character is their currently selected persona. All the characters that a player owns will be loaded into memory once they connect to the server. Characters are saved during a regular interval, and during specific events (e.g when the owning player switches away from one character to another).

They contain all information that is not persistent with the player; names, descriptions, model, currency, etc. For the most part, you'll want to keep all information stored on the character since it will probably be different or change if the player switches to another character. An easy way to do this is to use `ix.char.RegisterVar` to easily create accessor functions for variables that automatically save to the character object.

Functions

Character:AddBoost(boostID, attribID, boostAmount)

[View source »](#)

Temporarily increments one of this character's attributes. Useful for things like consumable items.

Parameters

- boostID [string](#)
Unique ID to use for the boost to remove it later
- attribID [string](#)
Name of the attribute to boost
- boostAmount [number](#)
Amount to increase the attribute by

Character:Ban(time)

[View source »](#)

Forces a player off their current character, and prevents them from using the character for the specified amount of time.

Parameters

- time number optional

Amount of seconds to ban the character for. If left as `nil`, the character will be banned permanently

Character:GetAttribute(key, default)

[View source »](#)

Returns the current value of an attribute. This is only valid on the server and owning client.

Parameters

- key [string](#)

Name of the attribute to get

- default number

Value to return if the attribute doesn't exist

Returns

- number

Value of the attribute

Character:GetBoost(attribID)

[View source »](#)

Returns all boosts that this character has for the given attribute. This is only valid on the server and owning client.

Parameters

- attribID [string](#)

Name of the attribute to find boosts for

Returns

- [table](#)

Table of boosts that this character has for the attribute

OR

- `nil`

If the character has no boosts for the given attribute

Character:GetBoosts()

[View source »](#)

Returns all boosts that this character has. This is only valid on the server and owning client.

Returns

- [table](#)

Table of boosts this character has

Character:GetClass()

[View source »](#)

Returns this character's current class.

Returns

- number

Index of the class this character is in

Character:GetCreateTime()

[View source »](#)

Returns the Unix timestamp of when this character was created (i.e the value of `os.time()` at the time of creation).

Returns

- number

Unix timestamp of when this character was created

Character:GetData(key, default)

[View source »](#)

Returns a data field set on this character. If it doesn't exist, it will return the given default or `nil`. This is only valid on the server and the owning client.

Parameters

- key [string](#)

Name of the field that's holding the data

- default

Value to return if the given key doesn't exist, or is `nil`

Returns

- any

Data stored in the field

OR

- nil

If the data doesn't exist, or is `nil`

Character:GetDescription()

[View source »](#)

Returns this character's physical description.

Returns

- `string`

This character's current description

Character:GetFaction()

[View source »](#)

Returns this character's faction.

Returns

- number

Index of the faction this character is currently in

Character:GetFlags()

[View source »](#)

Returns all of the flags this character has.

Returns

- `string`

Flags this character has represented as one string. You can access individual flags by iterating through the string letter by letter

Character:GetID()

[View source »](#)

Returns this character's database ID. This is guaranteed to be unique.

Returns

- number

Unique ID of character

Character:GetInventory()

[View source »](#)

Returns this character's associated [Inventory](#) object.

Returns

- [Inventory](#)

This character's inventory

Character:GetLastJoinTime()

[View source »](#)

Returns the Unix timestamp of when this character was last used by its owning player.

Returns

- number

Unix timestamp of when this character was last used

Character:GetModel()

[View source »](#)

Returns this character's model.

Returns

- [string](#)

This character's current model

Character:GetMoney()

[View source »](#)

Returns this character's money. This is only valid on the server and the owning client.

Returns

- number

Current money of this character

Character:GetName()

[View source »](#)

Returns this character's name

Returns

- [string](#)

This character's current name

Character:GetPlayer()

[View source »](#)

Returns the player that owns this character.

Returns

- player

Player that owns this character

Character:GetSchema()

[View source »](#)

Returns the schema that this character belongs to. This is useful if you are running multiple schemas off of the same database, and need to differentiate between them.

Returns

- [string](#)

Schema this character belongs to

Character:GetSteamID()

[View source »](#)

Returns the 64-bit Steam ID of the player that owns this character.

Returns

- [string](#)

Character:GiveFlags(flags)

[View source »](#)

Adds a flag to the list of this character's accessible flags. This does not overwrite existing flags.

Parameters

- flags [string](#)

Flag(s) this character should be given

Example Usage

```
character:GiveFlags("pet")
-- gives p, e, and t flags to the character
```

See Also

- [HasFlags](#)

Character:HasFlags(flags)

[View source »](#)

Returns `true` if the character has the given flag(s).

Parameters

- flags [string](#)

Flag(s) to check access for

Returns

- bool

Whether or not this character has access to the given flag(s)

Character:JoinClass(class)

[View source »](#)

Makes this character join a class. This automatically calls [KickClass](#) for you.

Parameters

- class number

Index of the class to join

Returns

- bool

Whether or not the character has successfully joined the class

Character:Kick()

[View source »](#)

Forces a player off their current character, and sends them to the character menu to select a character.

Character:KickClass()

[View source »](#)

Kicks this character out of the class they are currently in.

Character:RemoveBoost(boostID, attribID)

[View source »](#)

Removes a temporary boost from this character.

Parameters

- boostID [string](#)
Unique ID of the boost to remove
- attribID [string](#)
Name of the attribute that was boosted

Character:Save(callback)

[View source »](#)

Saves this character's info to the database.

Parameters

- callback function default: nil
Function to call when the save has completed.

Example Usage

```
ix.char.loaded[1]:Save(function()  
    print("done!")
```

```
end)
> done! -- after a moment
```

Character:SetAttrib(key, value)

[View source »](#)

Sets the value of an attribute for this character.

Parameters

- key [string](#)
Name of the attribute to update
- value number
New value for the attribute

Character:SetData(key, value)

[View source »](#)

Sets a data field on this character. This is useful for storing small bits of data that you need persisted on this character. This is networked only to the owning client. If you are going to be accessing this data field frequently with a getter/setter, consider using `ix.char.RegisterVar` instead.

Parameters

- key [string](#)
Name of the field that holds the data
- value
Any value to store in the field, as long as it's supported by GMod's JSON parser

Character:SetDescription(description)

[View source »](#)

Sets this character's physical description. This is automatically networked.

Parameters

- description [string](#)
New description for this character

Character:SetFaction(faction)

[View source »](#)

Sets this character's faction. Note that this doesn't do the initial setup for the player after the faction has been changed, so you'll have to update some character vars manually.

Parameters

- faction number
Index of the faction to transfer this character to

Character:SetFlags(flags)

[View source »](#)

Sets this character's accessible flags. Note that this method overwrites **all** flags instead of adding them.

Parameters

- flags [string](#)
Flag(s) this character is allowed to have

See Also

- [GiveFlags](#)

Character:SetModel(model)

[View source »](#)

Sets this character's model. This sets the player's current model to the given one, and saves it to the character. It is automatically networked.

Parameters

- model [string](#)
New model for the character

Character:SetMoney(money)

[View source »](#)

Sets this character's current money. Money is only networked to the player that owns this character.

Parameters

- money number

New amount of money this character should have

Character:SetName(name)

[View source »](#)

Sets this character's name. This is automatically networked.

Parameters

- name [string](#)

New name for the character

Character:Sync(receiver)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Networks this character's information to make the given player aware of this character's existence. If the receiver is not the owner of this character, it will only be sent a limited amount of data (as it does not need anything else). This is done automatically by the framework.

Parameters

- receiver [Player](#) default: nil

Player to send the information to. This will sync to all connected players if set to `nil`.

Character:TakeFlags(flags)

[View source »](#)

Removes this character's access to the given flags.

Parameters

- flags [string](#)

Flag(s) to remove from this character

Example Usage

```
-- for a character with "pet" flags
character:TakeFlags("p")
-- character now has e, and t flags
```


Character:UpdateAttrib(key, value)

[View source »](#)

Increments one of this character's attributes by the given amount.

Parameters

- key [string](#)
Name of the attribute to update
- value number
Amount to add to the attribute

Character:__eq(other)

[View source »](#)

Returns true if this character is equal to another character. Internally, this checks character IDs.

Parameters

- other [Character](#)
Character to compare to

Returns

- bool
Whether or not this character is equal to the given character

Example Usage

```
print(ix.char.loaded[1] == ix.char.loaded[2])  
> false
```

Character:__tostring()

[View source »](#)

Returns a string representation of this character

Returns

- [string](#)
String representation

Example Usage

```
print(ix.char.loaded[1])  
> "character[1]"
```

Entity

<https://docs.gethelix.co/classes/entity/>

Entity

Physical object in the game world.

Entities are physical representations of objects in the game world. Helix extends the functionality of entities to interface between Helix's own classes, and to reduce boilerplate code.

See the [Garry's Mod Wiki](#) for all other methods that the [Player](#) class has.

Functions

Entity:BlastDoor(velocity, lifeTime, bIgnorePartner)

[View source »](#)

Blasts a door off its hinges. Internally, this hides the door entity, spawns a physics prop with the same model, and applies force to the prop.

Parameters

- velocity vector

Velocity to apply to the door

- lifeTime number

How long to wait in seconds before the door is put back on its hinges

- bIgnorePartner boolean

Whether or not to ignore the door's partner in the case of double doors

Returns

- nil

If the entity is not a door

OR

- [Entity](#)

The physics prop created for the door

Entity:ClearNetVars(receiver)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Clears all of the networked variables.

Parameters

- receiver [table](#) default: nil

The players to clear the networked variable for

Entity:GetBlocker()

[View source »](#)

Returns the entity that is blocking this door from opening.

Returns

- [Entity](#)

Entity that is blocking this door

OR

- nil

If this entity is not a door, or there is no blocking entity

Entity:GetDoorPartner()

[View source »](#)

Returns the neighbouring door entity for double doors.

Returns

- [Entity](#)

This door's partner

OR

- nil

If the door does not have a partner

Entity:GetNetVar(key, default)

[View source »](#)

Retrieves a networked variable. If it is not set, it'll return the default that you've specified.

Parameters

- key [string](#)

Identifier of the networked variable

- default

Default value to return if the networked variable is not set

Returns

- any

Value associated with the key, or the default that was given if it doesn't exist

Example Usage

```
print(client:GetNetVar("example"))  
> Hello World!
```

See Also

- [SetNetVar](#)

Entity:IsChair()

[View source »](#)

Returns `true` if this entity is a chair.

Returns

- bool

Whether or not this entity is a chair

Entity:IsDoor()

[View source »](#)

Returns `true` if this entity is a door. Internally, this checks to see if the entity's class has `door` in its name.

Returns

- bool

Whether or not the entity is a door

Entity:IsLocked()

[View source »](#)

Returns `true` if the given entity is a button or door and is locked.

Returns

- `bool`

Whether or not this entity is locked; `false` if this entity cannot be locked at all (e.g not a button or door)

Entity:SendNetVar(key, receiver)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Sends a networked variable.

Parameters

- key `string`

Identifier of the networked variable

- receiver `table` default: `nil`

The players to send the networked variable to

Entity:SetNetVar(key, value, receiver)

[View source »](#)

Sets the value of a networked variable.

Parameters

- key `string`

Identifier of the networked variable

- value

New value to assign to the networked variable

- receiver `table` default: `nil`

The players to send the networked variable to

Example Usage

```
client:SetNetVar("example", "Hello World!")
```

See Also

- [GetNetVar](#)

Inventory

<https://docs.gethelix.co/classes/inventory/>

Inventory

Holds items within a grid layout.

Inventories are an object that contains [Items](#) in a grid layout. Every [Character](#) will have exactly one inventory attached to it, which is the only inventory that is allowed to hold bags - any item that has its own inventory (i.e a suitcase). Inventories can be owned by a character, or it can be individually interacted with as a standalone object. For example, the container plugin attaches inventories to props, allowing for items to be stored outside of any character inventories and remain "in the world".

You may be looking for the following common functions:

[Add](#) Which adds an item to the inventory.

[GetItems](#) Which gets all of the items inside the inventory.

[GetItemByID](#) Which gets an item in the inventory by it's item ID.

[GetItemAt](#) Which gets an item in the inventory by it's x and y

[GetID](#) Which gets the inventory's ID.

[HasItem](#) Which checks if the inventory has an item.

Functions

Inventory:Add(uniqueID, quantity, data, x, y, noReplication)

[View source »](#)

Add an item to the inventory.

Parameters

- uniqueID

The item unique ID (e.g `"handheld_radio"`) or instance ID (e.g `1024`) to add to the inventory

- quantity number default: 1

The quantity of the item to add

- data [table](#)

Item data to add to the item

- x number default: nil

The X position for the item

- y number default: nil

The Y position for the item

- noReplication boolean default: false

Whether or not the item's addition should not be replicated

Returns

- bool

Whether the add was successful or not

- [string](#)

The error, if applicable

OR

- number

The X position that the item was added to

- number

The Y position that the item was added to

- number

The inventory ID that the item was added to

Inventory:AddReceiver(client)

[View source](#) »

Adds a player as a receiver on this [Inventory](#). Receivers are players who will be networked the items inside the inventory.

Calling this will not automatically sync its current contents to the client. All future contents will be synced, but not anything that was not synced before this is called.

This function does not check the validity of `client`, therefore if `client` doesn't exist, it will error.

Parameters

- client [Player](#)

The player to add as a receiver.

Inventory:CanItemFit(x, y, w, h, item2)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Checks whether or not an [Item](#) can fit into the [Inventory](#) starting from `x` and `y`. Internally used by `FindEmptySlot`, in most cases you are better off using that. This function will search if all of the slots within `x + width` and `y + width` are empty, ignoring any space the [Item](#) itself already occupies.

Parameters

- `x` number

The beginning x coordinate to search for.

- `y` number

The beginning y coordiate to search for.

- `w` number

The [Item](#)'s width.

- `h` number

The [Item](#)'s height.

- `item2` [Item](#) default: `nil`

An [Item](#), if any, to ignore when searching.

Inventory:FindEmptySlot(w, h, onlyMain)

[View source »](#)

Finds an empty slot of a specified width and height. In most cases, to check if an [Item](#) can actually fit in the [Inventory](#), as if it can't, it will just return `nil`.

`FindEmptySlot` will loop through all the slots for you, as opposed to `CanItemFit` which you specify an `x` and `y` for. this will call `CanItemFit` anyway. If you need to check if an item will fit exactly at a position, you want `CanItemFit` instead.

Parameters

- `w` number

The width of the [Item](#) you are trying to fit.

- `h` number

The height of the [Item](#) you are trying to fit.

- onlyMain boolean

Whether or not to search any bags connected to this [Inventory](#)

Returns

- number

x The [x](#) coordinate that the [Item](#) can fit into.

- number

y The [y](#) coordinate that the [Item](#) can fit into.

OR

- number

x The [x](#) coordinate that the [Item](#) can fit into.

- number

y The [y](#) coordinate that the [Item](#) can fit into.

- [Inventory](#)

bagInv If the item was in a bag, it will return the inventory it was in.

See Also

- [CanItemFit](#)

Inventory:FindError()

[View source »](#)

Searches the inventory to find any stacked items. A common problem with developing, is that items will sometimes error out, or get corrupt. Sometimes, the server knows things you don't while developing live This function can be helpful for getting rid of those pesky errors.

Inventory:GetFilledSlotCount()

[View source »](#)

Returns the amount of slots currently filled in the Inventory.

Returns

- number

The amount of slots currently filled.

Inventory:GetID()

[View source »](#)

Returns this inventory's database ID. This is guaranteed to be unique.

Returns

- number

Unique ID of inventory

Inventory:GetItemAt(x, y)

[View source »](#)

Returns the item that currently exists within `x` and `y` in the [Inventory](#). Items that have a width or height greater than 0 occupy more than 1 `x` and `y`.

Parameters

- `x` number

The `x` coordindate to search in.

- `y` number

The `y` coordinate to search in.

Returns

- number

`x` The `x` coordinate that the [Item](#) is located at.

- number

`y` The `y` coordinate that the [Item](#) is located at.

Inventory:GetItemByID(id, onlyMain)

[View source »](#)

Get an item by it's specific Database ID.

Parameters

- `id` number

The ID to search for.

- `onlyMain` boolean

Whether or not to exclude bags that are present from the search.

Returns

- item

The item if it exists.

Inventory:GetItemCount(uniqueID, onlyMain)

[View source »](#)

Returns a count of a specific [Items](#) in the [Inventory](#)

Parameters

- uniqueID [string](#)

The Unique ID of the item.

- onlyMain boolean

Whether or not to exclude bags that are present from the search.

Returns

- number

The amount of [Items](#) this inventory has.

Example Usage

```
local curHighest, winner = 0, false
for client, character in ix.util.GetCharacters() do
    local itemCount = character:GetInventory():GetItemCount('water', false)
    if itemCount > curHighest then
        curHighest = itemCount
        winner = character
    end
end
-- Finds the thirstiest character on the server and returns their Character ID or false if no character has water.
```

Inventory:GetItems(onlyMain)

[View source »](#)

Returns a table of all the items that an [Inventory](#) has.

Parameters

- onlyMain boolean

Whether or not to exclude bags from this search.

Returns

- [table](#)

The items this [Inventory](#) has.

Inventory:GetItemsByBase(baseID, bOnlyMain)

[View source »](#)

Returns a table of [Items](#) by their base.

Parameters

- baseID [string](#)

The base to search for.

- bOnlyMain boolean

Whether or not to exclude bags that are present from the search.

Inventory:GetItemsByID(id, onlyMain)

[View source »](#)

Get a table of [Items](#) by their specific Database ID. It's important to note that while in 99% of cases, items will have a unique Database ID, developers or random GMod weirdness could cause a second item with the same ID to appear, even though, `ix.item.instances` will only store one of those. The inventory only stores a reference to the `ix.item.instance` ID, not the memory reference itself.

Parameters

- id number

The ID to search for.

- onlyMain boolean

Whether or not to exclude bags that are present from the search.

Returns

- item

The item if it exists.

Inventory:GetItemsByUniqueID(uniqueID, onlyMain)

[View source »](#)

Returns a table of all [Items](#) in the [Inventory](#) by their Unique ID. Not to be confused with [GetItemsByID](#) or [GetItemByID](#) which take in an Item Instance's ID instead.

Parameters

- `uniqueID` [string](#)

The Unique ID of the item.

- `onlyMain` `boolean`

Whether or not to exclude bags that are present from the search.

Returns

- `number`

The table of specified [Items](#) this inventory has.

Inventory:GetOwner()

[View source »](#)

Returns the player that owns this inventory.

Returns

- [Player](#)

Owning player

OR

- `nil`

If no connected player owns this inventory

Inventory:GetReceivers()

[View source »](#)

Get all of the receivers this [Inventory](#) has. Receivers are players who will be networked the items inside the inventory.

This function will automatically sort out invalid players for you.

Returns

- [table](#)

result The players who are on the server and allowed to see this table.

Inventory:GetShouldSave()

[View source »](#)

Gets whether or not an [Inventory](#) should save. Inventories that are marked to not save will not update in the Database, if they inventory is already saved, it will not be deleted when unloaded.

Returns

- bool

Returns the field `noSave` .

OR

- bool

Returns true if the field `noSave` is not registered to this inventory.

Inventory:GetSize()

[View source »](#)

Returns the grid size of this inventory.

Returns

- number

Width of inventory

- number

Height of inventory

Inventory:HasItem(targetID, data)

[View source »](#)

Returns the item with the given unique ID (e.g. `"handheld_radio"`) if it exists in this inventory. This method checks both this inventory, and any bags that this inventory has inside of it.

Parameters

- targetID [string](#)

Unique ID of the item to look for

- data [table](#) optional

Item data to check for

Returns

- [Item](#)

Item that belongs to this inventory with the given criteria

OR

- bool

`false` if the item does not exist

Example Usage

```
local item = inventory:HasItem("handheld_radio")

if (item) then
    -- do something with the item table
end
```

See Also

- [HasItems](#)
- [HasItemOfBase](#)

Inventory:HasItemOfBase(baseID, data)

[View source »](#)

Whether or not an [Inventory](#) has an item of a base, optionally with specified data. This function has an optional `data` argument, which will take a [table](#). it will match if the data of the item is correct or not.

Items which are a base will automatically have `base_` prefixed to their Unique ID, if you are having trouble finding your base, that is probably why.

Parameters

- baseID [string](#)

The Item Base's Unique ID.

- data [table](#) optional

The Item's data to compare against.

Returns

- item

The first [Item](#) of `baseID` that is found and there is no `data` argument or `data` was matched.

OR

- false

If no [Items](#) of `baseID` is found or the `data` argument, if specified didn't match.

Example Usage

```
local bHasWeaponEquipped = Entity(1):GetCharacter():GetInventory():HasItemOfBase('base_weapons', {'equip'} = true})
if bHasWeaponEquipped then
    Entity(1):Notify('One gun is fun, two guns is Woo-tastic.')
end
-- Notifies the player that they should get some more guns.
```

Inventory:HasItems(targetIDs)

[View source »](#)

Checks whether or not the [Inventory](#) has a table of items. This function takes a table with **no** keys and runs in order of first item > last item, this is due to the usage of the `#` operator in the function.

Parameters

- targetIDs [table](#)

A table of [Item](#) Unique ID's.

Returns

- bool

true Whether or not the [Inventory](#) has all of the items.

- [table](#)

targetIDs Your provided targetIDs table, but it will be empty.

OR

- bool

false

- [table](#)

targetIDs Table consisting of the items the [Inventory](#) did **not** have.

Example Usage

```
local itemFilter = {'water', 'water_sparkling'}
if not Entity(1):GetCharacter():GetInventory():HasItems(itemFilter) then return end
-- Filters out if this player has both a water, and a sparkling water.
```

Inventory:Initialize(id, width, height)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Initializes the inventory with the provided arguments.

Parameters

- id number
The [Inventory](#)'s database ID.
- width number
The inventory's width.
- height number
The inventory's height.

Inventory:Iter()

[View source »](#)

Returns an iterator that returns all contained items, a better way to iterate items than `pairs(inventory:GetItems())`

Returns

- function
iterator

Inventory:OnCheckAccess(client)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Checks whether a player has access to an inventory

Parameters

- client [Player](#)
Player to check access for

Returns

- bool

Whether or not the player has access to the inventory

Inventory:PrintAll()

[View source »](#)

Prints out the id, width, height, slots and each item in each slot of an [Inventory](#), used for debugging.

Inventory:Remove(id, bNoReplication, bNoDelete, bTransferring)

[View source »](#)

Removes an item from the inventory.

Parameters

- id number
The item instance ID to remove
- bNoReplication boolean default: false
Whether or not the item's removal should not be replicated
- bNoDelete boolean default: false
Whether or not the item should not be fully deleted
- bTransferring boolean default: false
Whether or not the item is being transferred to another inventory

Returns

- number
The X position that the item was removed from
- number
The Y position that the item was removed from

Inventory:RemoveReceiver(client)

[View source »](#)

The opposite of [AddReceiver](#). This function does not check the validity of `client`, therefore if `client` doesn't exist, it will error.

Parameters

- client [Player](#)

The player to remove from the receiver list.

Inventory:SendSlot(x, y, item)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Sends a specific slot to a character. This will not send all of the slots of the [Item](#) to the character, items can occupy multiple slots.

This will call `OnSendData` on the Item using all of the [Inventory](#)'s receivers.

This function should not be used to sync an entire inventory, if you need to do that, use [AddReceiver](#) and [Sync](#).

Parameters

- x number

The Inventory x position to send.

- y number

The Inventory y position to send.

- item [Item](#) optional

The item to send, if any.

See Also

- [AddReceiver](#)
- [Sync](#)

Inventory:SetOwner(owner, fullUpdate)

[View source »](#)

Sets the player that owns this inventory.

Parameters

- owner [Player](#)

The player to take control over the inventory.

- fullUpdate boolean

Whether or not to update the inventory immediately to the new owner.

Inventory:SetShouldSave(bNoSave)

[View source »](#)

Sets whether or not an [Inventory](#) should save. This will prevent an [Inventory](#) from updating in the Database, if the inventory is already saved, it will not be deleted when unloaded.

Parameters

- bNoSave boolean

Whether or not the Inventory should save.

Inventory:SetSize(width, height)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Sets the grid size of this inventory.

Parameters

- width number

New width of inventory

- height number

New height of inventory

Inventory:Sync(receiver)

[View source »](#)

Syncs the [Inventory](#) to the receiver. This will call Item.OnSendData on every item in the [Inventory](#).

Parameters

- receiver [Player](#)

The player to

Inventory:__tostring()

[View source »](#)

Returns a string representation of this inventory

Returns

- `string`

String representation

Example Usage

```
print(ix.item.inventories[1])  
> "inventory[1]"
```

Item

<https://docs.gethelix.co/classes/item/>

Item

Interactable entities that can be held in inventories.

Items are objects that are contained inside of an [Inventory](#), or as standalone entities if they are dropped in the world. They usually have functionality that provides more gameplay aspects to the schema. For example, the zipties in the HL2 RP schema allow a player to tie up and search a player.

For an item to have an actual presence, they need to be instanced (usually with `ix.item.Instance`). Items describe the properties, while instances are a clone of these properties that can have their own unique data (e.g an ID card will have the same name but different numerical IDs). You can think of items as the class, while instances are objects of the [Item](#) class.

Creating item classes ([ItemStructure](#))

Item classes are defined in their own file inside of your schema or plugin's `items/` folder. In these item class files you specify how instances of the item behave. This includes default values for basic things like the item's name and description, to more advanced things by overriding extra methods from an item base. See [ItemStructure](#) for information on how to define a basic item class.

Item classes in this folder are automatically loaded by Helix when the server starts up.

Item bases

If many items share the same functionality (i.e a can of soda and a bottle of water can both be consumed), then you might want to consider using an item base to reduce the amount of duplication for these items. Item bases are defined the same way as regular item classes, but they are placed in the `items/base/` folder in your schema or plugin. For example, a `consumables` base would be in `items/base/sh_consumables.lua`.

Any items that you want to use this base must be placed in a subfolder that has the name of the base you want that item to use. For example, for a bottled water item to use the consumable base, it must be placed in `items/consumables/sh_bottled_water.lua`. This also means that you cannot place items into subfolders as you wish, since the framework will try to use an item base that doesn't exist.

The default item bases that come with Helix are:

- `ammo` - provides ammo to any items with the `weapons` base
- `bags` - holds an inventory that other items can be stored inside of
- `outfit` - changes the appearance of the player that wears it
- `pacoutfit` - changes the appearance of the player that wears it using PAC3
- `weapons` - makes anySWEP into an item that can be equipped

These item bases usually come with extra values and methods that you can define/override in order to change their functionality. You should take a look at the source code for these bases to see their capabilities.

Item functions ([ItemFunctionStructure](#))

Requiring players to interact with items in order for them to do something is quite common. As such, there is already a built-in mechanism to allow players to right-click items and show a list of available options. Item functions are defined in your item class file in the `ITEM.functions` table. See [ItemFunctionStructure](#) on how to define them.

Helix comes with `drop`, `take`, and `combine` item functions by default that allows items to be dropped from a player's inventory, picked up from the world, and combining items together. These can be overridden by defining an item function in your item class file with the same name. See the `bags` base for example usage of the `combine` item function.

Item icons ([ItemIconStructure](#))

Icons for items sometimes don't line up quite right, in which case you can modify an item's `iconCam` value and line up the rendered model as needed. See [ItemIconStructure](#) for more details.

Functions

Item:Call(method, client, entity, ...)

[View source »](#)

Calls one of the item's methods.

Parameters

- method [string](#)
The method to be called
- client [Player](#)
The client to pass when calling the method, if applicable
- entity [Entity](#)
The eneity to pass when calling the method, if applicable
- ...
Arguments to pass to the method

Returns

- any
The values returned by the method

Item:GetCharacterID()

[View source »](#)

Returns the ID of the owning character, if one exists.

Returns

- number

The owning character's ID

Item:GetData(key, default)

[View source »](#)

Returns the value stored on a key within the item's data.

Parameters

- key [string](#)

The key in which the value is stored

- default default: nil

The value to return in case there is no value stored in the key

Returns

- any

The value stored within the key

Item:GetDescription()

[View source »](#)

Returns the description of the item.

Returns

- [string](#)

The description of the item

Item:GetEntity()

[View source »](#)

Returns the item's entity.

Returns

- entity

The entity of the item

Item:GetID()

[View source »](#)

Returns this item's database ID. This is guaranteed to be unique.

Returns

- number

Unique ID of item

Item:GetModel()

[View source »](#)

Returns the model of the item.

Returns

- [string](#)

The model of the item

Item:GetName()

[View source »](#)

Returns the name of the item.

Returns

- [string](#)

The name of the item

Item:GetOwner()

[View source »](#)

Returns the player that owns this item.

Returns

- player

Player owning this item

Item:GetPlayerID()

[View source »](#)

Returns the SteamID64 of the owning player, if one exists.

Returns

- number

The owning player's SteamID64

Item:GetSkin()

[View source »](#)

Returns the skin of the item.

Returns

- number

The skin of the item

Item:Hook(name, func)

[View source »](#)

Changes the function called on specific events for the item.

Parameters

- name [string](#)

The name of the hook

- func function

The function to call once the event occurs

Item:PostHook(name, func)

[View source »](#)

Changes the function called after hooks for specific events for the item.

Parameters

- name [string](#)

The name of the hook

- func function

The function to call after the original hook was called

Item:Print(detail)

[View source »](#)

A utility function which prints the item's details.

Parameters

- detail boolean default: false

Whether additional detail should be printed or not(Owner, X position, Y position)

Item:PrintData()

[View source »](#)

A utility function printing the item's stored data.

Item:Remove(bNoReplication, bNoDelete)

[View source »](#)

Removes the item.

Parameters

- bNoReplication boolean

Whether or not the item's removal should not be replicated.

- bNoDelete boolean

Whether or not the item should not be fully deleted

Returns

- bool

Whether the item was successfully deleted or not

Item:SetData(key, value, receivers, noSave, noCheckEntity)

[View source »](#)

Sets a key within the item's data.

Parameters

- key [string](#)

The key to store the value within

- value default: nil

The value to store within the key

- receivers [table](#) default: nil

The players to replicate the data on

- noSave boolean default: false

Whether to disable saving the data on the database or not

- noCheckEntity boolean default: false

Whether to disable setting the data on the entity, if applicable

Item:Spawn(position, angles)

[View source »](#)

Spawn an item entity based off the item table.

Parameters

- position vector

The position in which the item's entity will be spawned

- angles angle

The angles at which the item's entity will spawn

Returns

- entity

The spawned entity

Item:Transfer(invID, x, y, client, noReplication, isLogical)

[View source »](#)

Transfers an item to a specific inventory.

Parameters

- invID number

The inventory to transfer the item to

- x number

The X position to which the item should be transferred on the new inventory

- y number

The Y position to which the item should be transferred on the new inventory

- client [Player](#)

The player to which the item is being transferred

- noReplication boolean

Whether there should be no replication of the transferral

- isLogical boolean

Whether or not an entity should spawn if the item is transferred to the world

Returns

- bool

Whether the transfer was successful or not

- [string](#)

The error, if applicable

Item:__eq(other)

[View source »](#)

Returns true if this item is equal to another item. Internally, this checks item IDs.

Parameters

- other [Item](#)

Item to compare to

Returns

- bool

Whether or not this item is equal to the given item

Example Usage

```
print(ix.item.instances[1] == ix.item.instances[2])
> false
```

Item:__tostring()

[View source »](#)

Returns a string representation of this item.

Returns

- [string](#)

String representation

Example Usage

```
print(ix.item.instances[1])
> "item[1]"
```

Tables

ItemFunctionStructure

[View source »](#)

All item functions live inside of an item's `functions` table. An item function entry includes a few methods and fields you can use to customize the functionality and appearance of the item function. An example item function is below:

```
-- this item function's unique ID is "MyFunction"
ITEM.functions.MyFunction = {
    name = "myFunctionPhrase", -- uses the "myFunctionPhrase" language phrase when displaying in the UI
    tip = "myFunctionDescription", -- uses the "myFunctionDescription" language phrase when displaying in the UI
    icon = "icon16/add.png", -- path to the icon material
    OnRun = function(item)
        local client = item.player
        local entity = item.entity -- only set if this is function is being ran while the item is in the world

        if (IsValid(client)) then
            client:ChatPrint("This is a test.")

            if (IsValid(entity)) then
                client:ChatPrint(entity:GetName())
            end
        end

        -- do not remove this item from the owning player's inventory
    end
end
```



```

        return false
    end,
    OnCanRun = function(item)
        -- only allow admins to run this item function
        local client = item.player
        return IsValid(client) and client:IsAdmin()
    end
}

```

Fields

- name [string](#) optional

Language phrase to use when displaying this item function's name in the UI. If not specified, then it will use the unique ID of the item function

- tip [string](#) optional

Language phrase to use when displaying this item function's detailed description in the UI

- icon [string](#) optional

Path to the material to use when displaying this item function's icon

- OnRun function

Function to call when the item function is ran. This function is **ONLY** ran on the server.

The only argument passed into this function is the instance of the item being called. The instance will have its `player` field set if the item function is being ran by a player (which it should be most of the time). It will also have its `entity` field set if the item function is being ran while the item is in the world, and not in a player's inventory.

The item will be removed after the item function is ran. If you want to prevent this behaviour, then you can return `false` in this function. See the example above.

- OnCanRun function

Function to call when checking whether or not this item function can be ran. This function is ran **BOTH** on the client and server.

The arguments are the same as `OnCanRun`, and the `player` and `entity` fields will be set on the item instance accordingly. Returning `true` will allow the item function to be ran. Returning `false` will prevent it from running and additionally hide it from the UI. See the example above.

- OnClick function optional

This function is called when the player clicks on this item function's entry in the UI. This function is ran **ONLY** on the client, and is only ran if `OnCanRun` succeeds.

The same arguments from `OnCanRun` and `OnRun` apply to this function.

ItemIconStructure

[View source »](#)

Changing the way an item's icon is rendered is done by modifying the location and angle of the model, as well as the FOV of the camera. You can tweak the values in code, or use the `ix_dev_icon` console command to visually position the model and camera. An example entry for an item's icon is below:

```
ITEM.iconCam = {  
    pos = Vector(0, 0, 60),  
    ang = Angle(90, 0, 0),  
    fov = 45  
}
```

Note that this will probably not work for your item's specific model, since every model has a different size, origin, etc. All item icons need to be tweaked individually.

Fields

- pos vector

Location of the model relative to the camera. +X is forward, +Z is up

- ang angle

Angle of the model

- fov number

FOV of the camera

ItemStructure

[View source »](#)

When creating an item class, the file will have a global table `ITEM` set that you use to define the item's values/methods. An example item class is below:

`items/sh_brick.lua`

```
ITEM.name = "Brick"  
ITEM.description = "A brick. Pretty self-explanatory. You can eat it but you'll probably lose some teeth."  
ITEM.model = Model("models/props_debris/concrete_cynderblock001.mdl")  
ITEM.width = 1  
ITEM.height = 1  
ITEM.price = 25
```

Note that the below list only includes the default fields available for all items, and not special ones defined in custom item bases.

Fields

- name [string](#)

Display name of the item

- description [string](#)

Detailed description of the item

- model [string](#)

Model to use for the item's icon and when it's dropped in the world

- width number default: 1

Width of the item in grid cells

- height number default: 1

Height of the item in grid cells

- price number default: 0

How much money it costs to purchase this item in the business menu

- category [string](#) optional

Name of the category this item belongs to - mainly used for the business menu

- noBusiness boolean default: false

Whether or not to disallow purchasing this item in the business menu

- factions [table](#) optional

List of factions allowed to purchase this item in the business menu

- classes [table](#) optional

List of character classes allowed to purchase this item in the business menu. Classes are checked after factions, so the character must also be in an allowed faction

- flag [string](#) optional

List of flags (as a string - e.g. `"a"` or `"abc"`) allowed to purchase this item in the business menu. Flags are checked last, so the character must also be in an allowed faction and class

- iconCam [ItemIconStructure](#) optional

How to render this item's icon

- functions [table](#) optional

List of all item functions that this item has. See [ItemFunctionStructure](#) on how to define new item functions

Player

<https://docs.gethelix.co/classes/player/>

Player

Physical representation of connected player.

[Players](#) are a type of [Entity](#). They are a physical representation of a [Character](#) - and can possess at most one [Character](#) object at a time that you can interface with.

See the [Garry's Mod Wiki](#) for all other methods that the [Player](#) class has.

Functions

Player:CanShootWeapon()

[View source »](#)

Returns `true` if the player is able to shoot their weapon.

Returns

- `bool`
Whether or not the player can shoot their weapon

Player:ChatNotify(message)

[View source »](#)

Displays a notification for this player in the chatbox.

Parameters

- message [string](#)
Text to display in the notification

Player:ChatNotifyLocalized(message, ...)

[View source »](#)

Displays a notification for this player in the chatbox with the given language phrase.

Parameters

- message [string](#)

ID of the phrase to display to the player

- ...

Arguments to pass to the phrase

See Also

- [NotifyLocalized](#)

Player:CreateServerRagdoll(bDontSetPlayer)

[View source »](#)

Creates a ragdoll entity of this player that will be synced with clients. This does **not** affect the player like [SetRagdolled](#) does.

Parameters

- bDontSetPlayer boolean default: false

Whether or not to avoid setting the ragdoll's owning player

Returns

- entity

Created ragdoll entity

Player:DoStaredAction(entity, callback, time, onCancel, distance)

[View source »](#)

Performs a time-delay action that requires this player to look at an entity. If this player looks away from the entity before the action timer completes, the action is cancelled. This is usually used in conjunction with [SetAction](#) to display progress to the player.

Parameters

- entity [Entity](#)

that this player must look at

- callback function

Function to call when the timer completes

- time number

How much time in seconds this player must look at the entity for

- onCancel function default: nil

Function to call when the timer has been cancelled

- distance number default: 96

Maximum distance a player can move away from the entity before the action is cancelled

Example Usage

```
client:SetAction("Searching...", 4) -- for displaying the progress bar
client:DoStaredAction(entity, function()
    print("hello!")
end)
-- prints "hello!" after looking at the entity for 4 seconds
```

See Also

- [SetAction](#)

Player:ForceSequence(sequence, callback, time, bNoFreeze)

[View source »](#)

Forces this player's model to play an animation sequence. It also prevents the player from firing their weapon while the animation is playing.

Parameters

- sequence [string](#)

Name of the animation sequence to play

- callback function default: nil

Function to call when the animation finishes. This is also called immediately if the animation fails to play

- time number default: nil

How long to play the animation for. This defaults to the duration of the animation

- bNoFreeze boolean default: false

Whether or not to avoid freezing this player in place while the animation is playing

See Also

- [LeaveSequence](#)

Player:GetCharacter()

[View source »](#)

Returns this player's currently possessed [Character](#) object if it exists.

Returns

- [Character](#)

Currently loaded character

OR

- nil

If this player has no character loaded

Player:GetItemDropPos(entity)

[View source »](#)

Returns a good position in front of the player for an entity to be placed. This is usually used for item entities.

Parameters

- entity [Entity](#)

Entity to get a position for

Returns

- vector

Best guess for a good drop position in front of the player

Example Usage

```
local position = client:GetItemDropPos(entity)
entity:SetPos(position)
```

Player:GetLocalVar(key, default)

[View source »](#)

Retrieves a local networked variable. If it is not set, it'll return the default that you've specified. Locally networked variables can only be retrieved from the owning player when used from the client.

Parameters

- key [string](#)

Identifier of the local variable

- default

Default value to return if the local variable is not set

Returns

- any

Value associated with the key, or the default that was given if it doesn't exist

Example Usage

```
print(client:GetLocalVar("secret"))  
> 12345678
```

See Also

- [SetLocalVar](#)

Player:GetName()

[View source »](#)

Returns this player's current name.

Returns

- [string](#)

Name of this player's currently loaded character

OR

- [string](#)

Steam name of this player if the player has no character loaded

Player:GetPlayTime()

[View source »](#)

Returns the amount of time the player has played on the server.

Returns

- number

Number of seconds the player has played on the server

Player:IsFemale()

[View source »](#)

Returns `true` if the player currently has a female model. This checks if the model has `female`, `alyx` or `mossman` in its name, or if the player's model class is `citizen_female`.

Returns

- bool

Whether or not the player has a female model

Player:IsRestricted()

[View source »](#)

Returns `true` if the player is restricted - that is to say that they are considered "bound" and cannot interact with objects normally (e.g hold weapons, use items, etc). An example of this would be a player in handcuffs.

Returns

- bool

Whether or not the player is restricted

Player:IsRunning()

[View source »](#)

Returns `true` if the player is running. Running in this case means that their current speed is greater than their regularly set walk speed.

Returns

- bool

Whether or not the player is running

Player:IsStuck()

[View source »](#)

Whether or not this player is stuck and cannot move.

Returns

- bool

Whether or not this player is stuck

Player:IsWepRaised()

[View source »](#)

Returns `true` if the player has their weapon raised.

Returns

- bool

Whether or not the player has their weapon raised

Player:LeaveSequence()

[View source »](#)

Forcefully stops this player's model from playing an animation that was started by [ForceSequence](#).

Player:Notify(message)

[View source »](#)

Displays a prominent notification in the top-right of this player's screen.

Parameters

- message [string](#)

Text to display in the notification

Player:NotifyLocalized(message, ...)

[View source »](#)

Displays a notification for this player with the given language phrase.

Parameters

- message [string](#)

ID of the phrase to display to the player

- ...

Arguments to pass to the phrase

Example Usage

```
client:NotifyLocalized("mapRestarting", 10)
-- displays "The map will restart in 10 seconds!" if the player's language is set to English
```

See Also

- [ix.lang](#)

Player:PerformInteraction(time, entity, callback)

[View source »](#)

Performs a delayed action that requires this player to hold use on an entity. This is displayed to this player as a closing ring over their crosshair.

Parameters

- time number

How much time in seconds this player has to hold use for

- entity [Entity](#)

Entity that this player must be looking at

- callback function

Function to run when the timer completes. It will be ran right away if `time` is `0`. Returning `false` in the callback will not mark this interaction as dirty if you're managing the interaction state manually.

Player:RequestString(title, subTitle, callback, default)

[View source »](#)

Opens up a text box on this player's screen for input and returns the result. Remember to sanitize the user's input if it's needed!

Parameters

- title [string](#)

Title to display on the panel

- subTitle [string](#)

Subtitle to display on the panel

- callback function

Function to run when this player enters their input. Callback is ran with the user's input string.

- default [string](#) default: nil

Default value to put in the text box.

Example Usage

```
client:RequestString("Hello", "Please enter your name", function(text)
    client:ChatPrint("Hello, " .. text)
end)
-- prints "Hello, " in the player's chat
```

Player:ResetBodygroups()

[View source »](#)

Resets all bodygroups this player's model has to their defaults (`0`).

Player:SetAction(text, time, callback, startTime, finishTime)

[View source »](#)

Displays a progress bar for this player that takes the given amount of time to complete.

Parameters

- text [string](#)
Text to display above the progress bar
- time number default: 5
How much time in seconds to wait before the timer completes
- callback function
Function to run once the timer completes
- startTime number default: CurTime()
Game time in seconds that the timer started. If you are using `time` , then you shouldn't use this argument
- finishTime number default: startTime + time
Game time in seconds that the timer should complete at. If you are using `time` , then you shouldn't use this argument

Player:SetLocalVar(key, value)

[View source »](#)

Sets the value of a local networked variable.

Parameters

- key [string](#)
Identifier of the local variable
- value
New value to assign to the local variable

Example Usage

```
client:SetLocalVar("secret", 12345678)
```

See Also

- [GetLocalVar](#)

Player:SetRagdolled(bState, time, getUpGrace)

[View source »](#)

Sets this player's ragdoll status.

Parameters

- bState boolean
Whether or not to ragdoll this player
- time number default: 0
How long this player should stay ragdolled for. Set to 0 if they should stay ragdolled until they get back up manually
- getUpGrace number default: 5
How much time in seconds to wait before the player is able to get back up manually. Set to the same number as time to disable getting up manually entirely

Player:SetRestricted(bState, bNoMessage)

[View source »](#)

Sets this player's restricted status.

Parameters

- bState boolean
Whether or not to restrict this player
- bNoMessage boolean
Whether or not to suppress the restriction notification

Player:SetWepRaised(bState, weapon)

[View source »](#)

Sets whether or not this player's current weapon is raised.

Parameters

- bState boolean

Whether or not the raise the weapon

- weapon [Entity](#) default: GetActiveWeapon()

Weapon to raise or lower. You should pass this argument if you already have a reference to this player's current weapon to avoid an expensive lookup for this player's current weapon.

Player:SyncVars()

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Synchronizes networked variables to the client.

Player:ToggleWepRaised()

[View source »](#)

Inverts this player's weapon raised state. You should use [SetWepRaised](#) instead of this if you already have a reference to this player's current weapon.

ixLabel

Alternative to DLabel that adds extra functionality.

This panel is meant for drawing single-line text. It can add extra kerning (spaces between letters), and it can forcefully scale the text down to fit the current width, without cutting off any letters. Text scaling is most useful when docking this panel without knowing what the width could be. For example, text scaling is used for the character name in the character status menu.

```
local label = vgui.Create("ixLabel")
label:SetText("hello world")
label:SetFont("ixMenuButtonHugeFont")
label:SetContentAlignment(5)
label:SetText(Color(255, 255, 255, 255))
label:SetBackgroundColor(Color(200, 30, 30, 255))
label:SetPadding(8)
label:SetScaleWidth(true)
label:SizeToContents()
```

Functions

ixLabel:CalculateAlignment(width, height, textWidth, textHeight)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Returns the X and Y location of the text taking into account the text alignment and padding.

Parameters

- width number
Width of the panel
- height number
Height of the panel
- textWidth number

Width of the text

- textHeight number

Height of the text

Returns

- number

X location to draw the text

- number

Y location to draw the text

ixLabel:DrawKernedText(width, height)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Draws the current text with the current kerning.

Parameters

- width number

Width of the panel

- height number

Height of the panel

ixLabel:DrawText(width, height)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Draws the current text.

Parameters

- width number

Width of the panel

- height number

Height of the panel

ixLabel:GetBackgroundColor()

[View source »](#)

Returns the current background color for this panel.

Returns

- color

Current background color

ixLabel:GetContentAlignment()

[View source »](#)

Returns the current content alignment for this panel.

Returns

- number

Current content alignment

ixLabel:GetContentSize(bCalculate)

[View source »](#)

Returns the size of the text, taking into account the current kerning.

Parameters

- bCalculate boolean default: false

Whether or not to recalculate the content size instead of using the cached copy

Returns

- number

Width of the text

- number

Height of the text

ixLabel:GetFont()

[View source »](#)

Returns the current font for this panel.

Returns

- [string](#)

Name of current font

ixLabel:GetKerning()

[View source »](#)

Returns the current kerning for this panel.

Returns

- number

Current kerning

ixLabel:GetPadding()

[View source »](#)

Returns how much padding this panel has around its text.

Returns

- number

Current padding

ixLabel:GetScaleWidth()

[View source »](#)

Returns whether or not this panel will scale its text down to fit its width.

Returns

- bool

Whether or not this panel will scale its text

ixLabel:GetText()

[View source »](#)

Returns the current text for this panel.

Returns

- `string`

Current text

ixLabel:GetTextColor()

[View source »](#)

Returns the current text color for this panel.

Returns

- `color`

Current text color

ixLabel:SetBackgroundColor(color)

[View source »](#)

Sets the color of the background to draw behind the text.

Parameters

- `color color`

New color to use

ixLabel:SetContentAlignment(alignment)

[View source »](#)

Changes how the text is aligned when drawing. Valid content alignment values include numbers `1` through `9`. Each number's corresponding alignment is based on its position on a numpad. For example, `1` is bottom-left, `5` is centered, `9` is top-right, etc.

Parameters

- `alignment number`

Alignment to use

ixLabel:SetDropShadow(distance, color)

[View source »](#)

Sets the drop shadow to draw behind the text.

Parameters

- distance number

How far away to draw the shadow in pixels. Set to 0 to disable

- color optional

Color of the shadow. Defaults to a dimmed version of the text color

ixLabel:SetFont(font)

[View source »](#)

Sets the font used to draw the text.

Parameters

- font [string](#)

Name of the font to use

ixLabel:SetKerning(kerning)

[View source »](#)

Sets the spacing between each character of the text in pixels. Set to 0 to disable. Kerning is disabled by default.

Parameters

- kerning number

How far apart to draw each letter

ixLabel:SetPadding(padding)

[View source »](#)

How much spacing to use around the text when its drawn. This uses uniform padding on the top, left, right, and bottom of this panel.

Parameters

- padding number

Padding to use

ixLabel:SetScaleWidth(bScale)

[View source »](#)

Whether or not to scale the width of the text down to fit the width of this panel, if needed.

Parameters

- bScale boolean

Whether or not to scale

ixLabel:SetText(text)

[View source »](#)

Sets the text for this label to display.

Parameters

- text [string](#)

Text to display

ixLabel:SetTextColor(color)

[View source »](#)

Sets the color of the text to use when drawing.

Parameters

- color color

New color to use

[View source »](#)

Sets the size of the panel to fit the content size with the current padding. The content size is recalculated when this method is called.

ixTooltip

Generic information panel.

Tooltips are used extensively throughout Helix: for item information, character displays, entity status, etc. The tooltip system can be used on any panel or entity you would like to show standardized information for. Tooltips consist of the parent container panel ([ixTooltip](#)), which is filled with rows of information (usually [ixTooltipRow](#), but can be any docked panel if non-text information needs to be shown, like an item's size).

Tooltips can be added to panel with `panel:SetHelixTooltip()`. An example taken from the scoreboard:

```
panel:SetHelixTooltip(function(tooltip)
    local name = tooltip:AddRow("name")
    name:SetImportant()
    name:SetText(client:SteamName())
    name:SetBackgroundColor(team.GetColor(client:Team()))
    name:SizeToContents()

    tooltip:SizeToContents()
end)
```

Functions

ixTooltip:AddRow(id)

[View source »](#)

Creates a new [ixTooltipRow](#) panel and adds it to the bottom of this tooltip.

Parameters

- id [string](#)

Name of the new row. This is used to reorder rows if needed

Returns

- panel

Created row

ixTooltip:AddRowAfter(after, id)

[View source »](#)

Creates a new [ixTooltipRow](#) and adds it after the row with the given `id`. The order of the rows is set via setting the Z position of the panels, as this is how VGUI handles ordering with docked panels.

Parameters

- after [string](#)
Name of the row to insert after
- id [string](#)
Name of the newly created row

Returns

- panel
Created row

ixTooltip:GetCursorPosition()

[View source »](#)

Returns the current position of the mouse cursor on the screen.

Returns

- number
X position of cursor
- number
Y position of cursor

ixTooltip:GetRow(id)

[View source »](#)

Returns an [ixTooltipRow](#) corresponding to the given name.

Parameters

- id [string](#)
Name of the row

Returns

- panel

Corresponding row

OR

- nil

If the row doesn't exist

ixTooltip:IsMinimal()

[View source »](#)

Whether or not this tooltip should be displayed in a minimal format.

Returns

- bool

True if this tooltip should be displayed in a minimal format

See Also

- [ixTooltipRow:IsMinimal](#)

ixTooltip:SetEntity(entity)

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Sets the entity associated with this tooltip. Note that this function is not how you get entities to show tooltips.

Parameters

- entity [Entity](#)

Entity to associate with this tooltip

[View source »](#)

Resizes the tooltip to fit all of the child panels. You should always call this after you are done adding all of your rows.

ixTooltipRow

<https://docs.gethelix.co/panels/ixtooltiprow/>

ixTooltipRow

Text container for **ixTooltip**.

Rows are the main way of interacting with **ixTooltips**. These derive from **DLabel** panels, which means that making use of this panel will be largely the same as any DLabel panel.

Functions

ixTooltipRow:IsMinimal()

[View source »](#)

Whether or not this tooltip row should be displayed in a minimal format. This usually means no background and/or smaller font. You probably won't need this if you're using regular **ixTooltipRow** panels, but you should take into account if you're creating your own panels that derive from **ixTooltipRow**.

Returns

- bool

True if this tooltip row should be displayed in a minimal format

ixTooltipRow:Paint(width, height)

[View source »](#)

Called when the foreground of this row should be painted. If you are overriding this in a subclassed panel, make sure you call **ixTooltipRow:PaintBackground** at the beginning of your function to make its style consistent with the rest of the framework.

Parameters

- width number

Width of the panel

- height number

Height of the panel

ixTooltipRow:PaintBackground(width, height)

[View source »](#)

Called when the background of this row should be painted. This will paint the background with the `DrawImportantBackground` function set in the skin by default.

Parameters

- width number
Width of the panel
- height number
Height of the panel

ixTooltipRow:SetBackgroundColor(color)

[View source »](#)

Sets the background color of this row. This should be used sparingly to avoid overwhelming players with a bunch of different colors that could convey different meanings.

Parameters

- color color
New color of the background. The alpha is clamped to 100-255 to ensure visibility

ixTooltipRow:SetImportant()

[View source »](#)

Sets this row to be more prominent with a larger font and more noticable background color. This should usually be used once per tooltip as a title row. For example, item tooltips have one "important" row consisting of the item's name. Note that this function is a fire-and-forget function; you cannot revert a row back to it's regular state unless you set the font/colors manually.

[View source »](#)

Resizes this panel to fit its contents. This should be called after setting the text.

Internal

This is an internal function! You are able to use it, but you risk unintended side effects if used incorrectly.

[View source »](#)

Resizes the height of this panel to fit its contents.